



תיק פרויקט

שם בית הספר: תיכון אלון רמת השרון

שם הפרויקט: בניית מערכת לניהול משימות

שם התלמיד: נועם דסה

ת.ז.: 216431114

שמות המנחים: עידן פלדברג ועודי רז

שם החלופה: הגנת סייבר ומערכות הפעלה

תאריך הגשה: 30.4.2025



Sunday.com

The future of the task management

תוכן עניינים

4	מבוא	
4	ייזום	○
4	תיאור ראשוני של המערכת	○
4	הגדרת הלקוח	○
4	הגדרת יעדים/מטרות	○
4	הגדרת התועלות הצפויות	○
4	סקירת פתרונות	○
5	סקירת טכנולוגית	○
5	תיחום הפרויקט	○
5	פירוט תיאור המערכת (אפיון):	○
5	תיאור מפורט יותר של המערכת	○
5	פירוט מרכיבים שיש לקחת בחשבון במערכת	○
5	פירוט הבדיקות	○
7	תיאור תחום הידע	
7	פירוט יכולות צד שרת מסד נתונים	○
8	פירוט יכולות צד שרת מרכזי (Flask)	○
9	פירוט יכולות צד שרת (ConnectionWithDatababse)	○
9	פירוט יכולות צד לקוח	○
10	ארכיטקטורה של הפרויקט	
10	תיאור הארכיטקטורה של המערכת המוצעת	○
13	תיאור הטכנולוגיה הרלוונטית	○
14	תיאור זרימת המידע במערכת	○
14	תיאור האלגוריתמים מרכזים בפרויקט	○
15	תיאור אלגוריתמים קיימים לפתרון הבעיה	○
16	תיאור סביבת הפיתוח	○
16	תיאור פרוטוקול התקשורת	○
21	תיאור מסכי מערכת	○
25	תיאור מסד הנתונים	○
26	סקירת חולשות והאיומים	○
26	שכבת האפליקציה:	
26	שכבת התעבורה:	
27	מימוש הפרויקט	
27	חלק א' -	
28	שם המחלקה: ConnectionWithDatabase	
28	שם המחלקה: Network	

29	שם המחלקה: DatabaseParent
30	שם המחלקה: DatabaseClients
32	חלק ב' –
40	חלק ג' –
40	מסמך בדיקות
40	מדריך למשתמש
40	עץ קבצים
40	WOWDatabase side:
40	WOWProject:
41	התקנת המערכת
41	משתמשי המערכת
41	צילומי מסכי הפרויקט הרלוונטיים
41	רפלקציה
42	ביבליוגרפיה
43	נספחים

מבוא

○ ייזום

○ תיאור ראשוני של המערכת

המערכת שאני מפתח היא פלטפורמה מודרנית לניהול פרויקטים, מבוססת על יישום בדפדפן (Web application) ולתקשורת בזמן אמת. המערכת כוללת הרשאות, ניהול משימות, ניהול sprints, חברים, מערכת שיתוף, מערכת הודעות והתראות.

בחרתי בפרויקט זה כי הוא מאפשר לי לבנות מערכת המודרנית עם לוגיקה מורכבת תוך שימוש בטכנולוגיות מודרניות. זהו פרויקט מאתגר שמעודד פיתוח ארכיטקטורה המתאימה לניהול נתונים, ושליטה בהרשאות – דברים שחשובים בעולם המודרני.

בנוסף, הוא נותן לי הזדמנות ללמוד שפות חדשות שלא העמקתי בעבר המשלבות את עולם התכנות, הארגון והעיצוב. המערכת נותנת מענה לצורך ממשי בארגון משימות, ומאפשרת לי לבטא את היכולות הטכניות שלי תוך בניית כלי שימושי.

רציונל לפרויקט – בעולם העבודה המודרני, ניהול פרויקטים יעיל הוא צורך בסיסי בכל ארגון. ריבוי משימות, תיאום בין צוותים, מעקב אחר זמנים ועמידה ביעדים מחייבים פתרון טכנולוגי מתקדם, נוח ואינטואיטיבי. מתוך צורך זה עלה הרעיון לפתח מערכת ניהול משימות דיגיטלית, שתאפשר לארגונים ובעלי עסקים לנהל תהליכי עבודה בצורה מסודרת, שקופה ודינמית.

המערכת נועדה לשמש כפלטפורמה חכמה לניהול משימות ותקשורת פנים-ארגונית. באמצעותה, משתמשים יוכלו ליצור לוחות (sprints) להוסיף משימות, להקצות עובדים, להגדיר סטטוסים, לתעדף פעולות ולעקוב אחר התקדמות בזמן אמת. יתרה מכך, תזכורות והתראות, שיאפשרו חיסכון בזמן וצמצום טעויות אנוש.

○ הגדרת הלקוח

חברים, משפחה, תלמידים, מורים או כל אדם שזקוק לכלי ניהול פרויקטים, כולל צוותים קטנים או עצמאיים.

○ הגדרת יעדים/מטרות

- לבנות מערכת נוחה לשימוש הכוללת תצוגת משימות, משתמשים וסטטוס של כל משימה.
- לספק דרך פשוטה למשתמש להוסיף, לערוך ולמחוק משימות.
- ליצור ממשק ניהול משתמשים והרשאות לפי תפקידים (למשל, מנהל, עובד, אורח).
- יצירת ממשק גרפי מקצועי ואינטואיטיבי.
- תמיכה במצבי תצוגה שונים (כגון מצב לילה, שפה).
- ממשק ידידותי ונגיש – עיצוב ממשק מודרני שמותאם למחשב, עם דגש על חווית משתמש חיובית.
- התראות ואוטומציה – שילוב של מנגנוני התראה ואוטומציות חכמות להפחתת טעויות אנוש וייעול העבודה השוטפת.

○ הגדרת התועלות הצפויות

ניהול משימות, ממשק משתמש נוח, הרשאות לפי תפקידים, מערכת צ'אט, תמיכה בתצוגות שונות, אבטחת גישה לפי הרשאות.

○ סקירת פתרונות

קיימות מספר מערכות דומות כמו Monday.com, Trello, Asana, ClickUp ו-Sunday.com שמספקות פתרונות לניהול פרויקטים. כל אחת מהן מציעה יתרונות כמו אינטגרציה עם כלים חיצוניים, ממשק משתמש מושקע וניהול מתקדם של משימות. עם זאת:

- Trello פשוט מידי ואינו מותאם לצוותים גדולים או פרויקטים מורכבים.
- Asana דורשת תשלום עבור חלק מהפיצ'רים החשובים.

- ClickUp עשיר בפיצ'רים אך לעיתים מסורבל בשימוש
- המערכות לרוב מתמקדות באוטומציות או תצוגות, ופחות גמישות בהתאמה אישית לצרכים של צוותים קטנים/סטודנטים.

המערכת שאני מפתח שואפת לשלב פשטות ונגישות עם אפשרויות מתקדמות ולהתמקד בצרכים של צוותים קטנים.

○ סקירת טכנולוגית

בפרויקט השתמשתי בטכנולוגית Backend ו-FrontEnd. בחלק ה-FrontEnd השתמשתי ב-HTML/CSS/JavaScript בחלק ה-Backend השתמשתי בשפת Python עם טכנולוגית Flask. בנוסף גם RestAPI. בתהליך הקישור בין צד ה-FrontEnd לבין ה-Backend השתמשתי ב-WebSocket. לאחסון הנתונים השתמשתי במסד נתונים מסוג SQLite בשימוש בשפת SQL.

○ תיחום הפרויקט

פרויקט עוסק בתחום רשתות תקשורת. מיועדת לפעול על מחשב אישי או רשת מקומית. המשתמשים יתחברו דרך הממשק לפי ההרשאות.

○ פירוט תיאור המערכת (אפיון):

- תיאור מפורט יותר של המערכת
- ניהול משימות, ספרינטים (קבוצת משימות), משתמשים, הרשאות, שיתוף, הודעות, ממשק משתמש, תצוגות שונות ואבטחה.

○ פירוט מרכיבים שיש לקחת בחשבון במערכת

רכיב	תיאור
ממשק משתמש (Frontend)	HTML, CSS, JS - הצגת משימות, משתמשים, תצוגות, צ'אט והגדרות.
שרת צד אחורי (Backend)	מבוסס Flask - אחראי על ניהול נתונים, הרשאות APIs, ותקשורת.
מסד נתונים	שמירת משימות, פרויקטים, חברים, משתמשים, צ'אטים, הרשאות ועוד (למשל SQLite)
Socket.IO	ניהול תקשורת בזמן אמת - בעיקר בצ'אט ובמערכת ההתראות.
מערכת הרשאות	הגדרת תפקידים שונים (מנהל, עובד, משתמש) עם גישה שונה לכל תפקיד
מערכת הודעות והתראות	שליחת עדכונים בזמן אמת למשתמשים.
תמיכה במצבים שונים	לדוגמה: מצב לילה, שינוי שפה, התאמה לנייד/דסקטופ.
אבטחה	הגנה על נתונים, הרשאות לפי גישה, חסימת גישה לא מורשית

○ פירוט הבדיקות

כדי לוודא שהמערכת פועלת כשורה, נבצע:

- **בדיקות יחידה (Unit Tests)** - עבור הפונקציות המרכזיות (ניהול משימה, עריכת פרויקטים).
- **בדיקות אינטגרציה** - בדיקה של תקשורת בין ה-Backend ל-Frontend כולל Socket.IO לצ'אט.
- **בדיקות ממשק משתמש** - ווידאו שהממשק ברור, מגיב נכון וידידותי.
- **בדיקות הרשאות** - לוודא שכל תפקיד מקבל גישה רק למה שהוא אמור.

○ תכנון וניהול לוי"ז

תאריך	תיאור	שלב פיתוח
8.12.2024	הגדרת הפיצ'רים	אפיון המערכת
9.12.2024	הקמת השרת, בסיס נתונים, תקשורת באמצעות socket	BackEnd
15.12.2024	יצירת הממשק הגרפי, תצוגת משימות, ניהול משתמשים	FrontEnd
2.1.2025	אינטגרציה מלאה בין השרת לממשק.	חיבור בין הממשקים
10.2.2025	הרצת בדיקות על כל רכיבי המערכת.	בדיקות ותיקונים
28.2.2025	הוספת עיצוב, תצוגות שונות	שיפורים וסיום

○ ניהול הסיכונים וההזדמנויות אם קיימים

סיכון	דרכי התמודדות
תקשורת בין חלקי המערכת	שימוש בכלים מתועדים היטב כמו Flask + Socket.IO
אבטחת מידע	הצפנת המידע העובר בתקשורת וגם במסד הנתונים באמצעות טיפול ב-SQL Injection

תיאור תחום הידע

○ פירוט יכולות צד שרת מסד נתונים

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות	אובייקטים נחוצים
חיבור למסד נתונים	פתיחת חיבור לשרת נתונים בהתבסס על session ID	- התחברות - פתיחת סוקט TCP - ניהול זיהוי	socket, session_id
שליחת פקודות	שליחת פקודות בצורה טקסטואלית לשרת	- חיבור פתוח - קידוד נתונים - תבנית פקודות	send(), פרוטוקול פקודות, תקשורת, הצפנה
קבלת תגובה	קבלת תשובה מהשרת בפורמט תקין	- קבלת נתונים משרת המרכזי - קריאת מחרוזת - טיפול בנתונים	JSON, receive(), טקסט רגיל, תקשורת, הצפנה
יצירת חשבון	טיפול ביצירת חשבון ללקוח חדש	- קבלת נתונים משרת המרכזי - פענוח - בדיקה במסד נתונים עבור כפילות פרטים - הוספת משתמש למסד נתונים	תקשורת, הצפנה
כניסה לחשבון	הכנסת המשתמש למערכת	- קבלת נתונים משרת המרכזי - פענוח - בדיקה במסד נתונים עבור כפילות פרטים - עדכון משתמש במסד נתונים	תקשורת, הצפנה
הוספת פרויקט	הוספת פרויקט חדש למסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח - הוספת פרויקט למסד נתונים	תקשורת, הצפנה
שכפול פרויקט	שכפול פרויקט קיים עבור משתמש	- קבלת בקשת שכפול משרת המרכזי - פענוח - העתקת נתונים בטבלאות רלוונטיות במסד הנתונים	תקשורת, הצפנה
עדכון פרויקט	עדכון שם הפרויקט ותיאור הפרויקט במסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח - עדכון הפרטים במסד הנתונים - תקינות הסמלים בתיאור הפרויקט (" ; ")	תקשורת, הצפנה, ast
מחיקת פרויקט	מחיקת פרויקט ממסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח - עדכון הפרטים במסד הנתונים	תקשורת, הצפנה
שיתוף פרויקט	שיתוף הפרויקט הקיים עבור חברו	- קבלת נתונים משרת המרכזי - פענוח - עדכון הפרטים במסד הנתונים	תקשורת, הצפנה
הוספה/עדכון משימות	הוספת או עדכון משימות לפרויקט במסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח - עדכון הפרטים במסד הנתונים	תקשורת, הצפנה, JSON
ניהול ספרינטים	הוספה, עריכה, מחיקה של ספרינטים במסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח	תקשורת, הצפנה, JSON

ניהול חברים	שמירת שליחת בקשה, אישור חברות במסד הנתונים	- קבלת נתונים משרת המרכזי - פענוח - עדכון הפרטים במסד הנתונים	תקשורת, הצפנה, JSON
-------------	--	---	---------------------

○ פירוט יכולות צד שרת מרכזי (Flask)

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות	אובייקטים נחוצים
יצירת חשבון	טיפול ביצירת חשבון ללקוח חדש	- קבלת נתונים מלקוח - הצפנה - בדיקה במסד נתונים עבור כפילות פרטים	תקשורת, פענוח, מסד נתונים
כניסה לחשבון	הכנסת המשתמש למערכת	- קבלת נתונים מלקוח - הצפנה - הוספת פרטים נוספים ל-session	תקשורת, פענוח, מסד נתונים
צפייה בפרויקטים	שליפת פרויקטים מהשרת	- שליחת בקשה לשרת נתונים - הצפנה - פענוח תגובת JSON	תקשורת, פענוח, session, JSON
הוספת פרויקט	יצירת פרויקט חדש עבור המשתמש	- קבלת נתונים מהלקוח - הצפנה - שליחה לשרת נתונים - עדכון ב-session	מסד נתונים, פענוח, JSON, session, מסד נתונים
שכפול פרויקט	העתקת פרויקט קיים עם מזהה חדש	- קבלת נתונים מלקוח - שליפה מ-session - הצפנה - שליחה לשרת נתונים - קבלת תגובה	מסד נתונים, תקשורת, פענוח, session, JSON
עריכת הפרויקט	עריכת שם הפרויקט ותיאור הפרויקט	- קבלת נתונים מלקוח - הצפנה - עדכון הפרטים במסד הנתונים - עדכון session	תקשורת, פענוח, session, JSON
מחיקת פרויקט	הסרת פרויקט קיים	- קבלת מזהה פרויקט - הצפנה - שליחה לשרת נתונים - עדכון session	תקשורת, session, פענוח, מסד נתונים
ניהול משימות	הוספה, עריכה, מחיקה ועדכון של משימות בפרויקט	- קבלת נתונים מלקוח - ניהול - JSON - הצפנה - עדכון session - שליחה לשרת נתונים	תקשורת, JSON, session, פענוח, מסד נתונים
ניהול ספרינטים	הוספה, עריכה, מחיקה של ספרינטים	- קבלת נתונים מלקוח - ניהול צבעים - קידוד JSON - הצפנה - שליחה לשרת נתונים	תקשורת, JSON, session, פענוח, מסד נתונים
שליחת בקשת חברות	שליחת בקשת חברות למשתמש אחר	- קבלת נתונים מלקוח - קבלת מייל - אימות - הצפנה - שליחה לשרת נתונים	session, socketio, JSON, פענוח, מסד נתונים

	• שיטור Real-Time		
• קבלת נתונים מלקוח • אימות סיסמה • הצפנה • שליחה לשרת נתונים	• session, תקשורת, socketio, פענוח, מסד נתונים	טיפול בבקשת חברות שהתקבלה	אישור/דחיית בקשה
• שמירת projects, session_id, client_id ועוד	• Flask-Session, UUID, cookies	ניהול נתוני session לאורך זמן	ניהול session
• התחברות • הצטרפות לחדרים • שליחת שידורים	• socketio, room, event handlers	שידורים והתראות בזמן אמת	תקשורת WebSocket

○ פירוט יכולות צד שרת (ConnectionWithDatabase)

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות	אובייקטים נחוצים
התחברות למסד נתונים	פתיחת חיבור דרך socket לשרת	• יצירת סוקט TCP • התחברות לכתובת ה-IP והפורט • בדיקת התחברות מוצלחת	IP, socket, כתובת, ופורט
שליחת הודעה מוצפנת	שליחת מידע לאחר הצפנה	• הצפנה • חישוב אורך ההודעה • שליחה עם קידוד באורך 4 בתים	socket, פענוח
קבלת הודעה מהשרת	קבלת מידע, בדיקת ניתוק ופענוח	• קריאת הודעה מהשרת • בדיקת ניתוק • חיתוך אורך ההודעה • פענוח	socket, הצפנה
סגירת חיבור	סגירה מסודרת של החיבור	• בדיקת סטטוס התחברות • סגירת socket • טיפול בשגיאה אפשרית במהלך כיבוי	socket

○ פירוט יכולות צד לקוח

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות	אובייקטים נחוצים
הרשמה למערכת	רשום משתמש חדש במערכת – קליטת פרטים אישיים נדרשים	○ ממשק משתמש – מסך הרשמה ○ קליטת נתונים מלקוח ○ בדיקת התקינות ○ עברת נתונים לשרת המרכזי ○ הוספת משתמש ל-session	ממשק משתמש, Java Script, HTML, CSS
התחברות למערכת	התחברות עם שם משתמש וסיסמה	• ממשק משתמש – מסך התחברות • קליטת נתונים מלקוח • בדיקת התקינות • עברת נתונים לשרת המרכזי • הוספת משתמש ל-session (במקרה שלא נרשם לפני)	HTML, CSS, JavaScript, JSON, session
הצגת פרויקטים	הצגת רשימת פרויקטים + משותפים	• שליפת נתונים מהשרת • עיבוד JSON • הצגה בדף	fetch/ajax, JSON, DOM
עריכת פרויקט	UI לעריכת פרויקט קיים ושליחתו לשרת	• טופס עריכה • שליחה לשרת באמצעות fetch • עדכון ברשימה	JavaScript, DOM, API

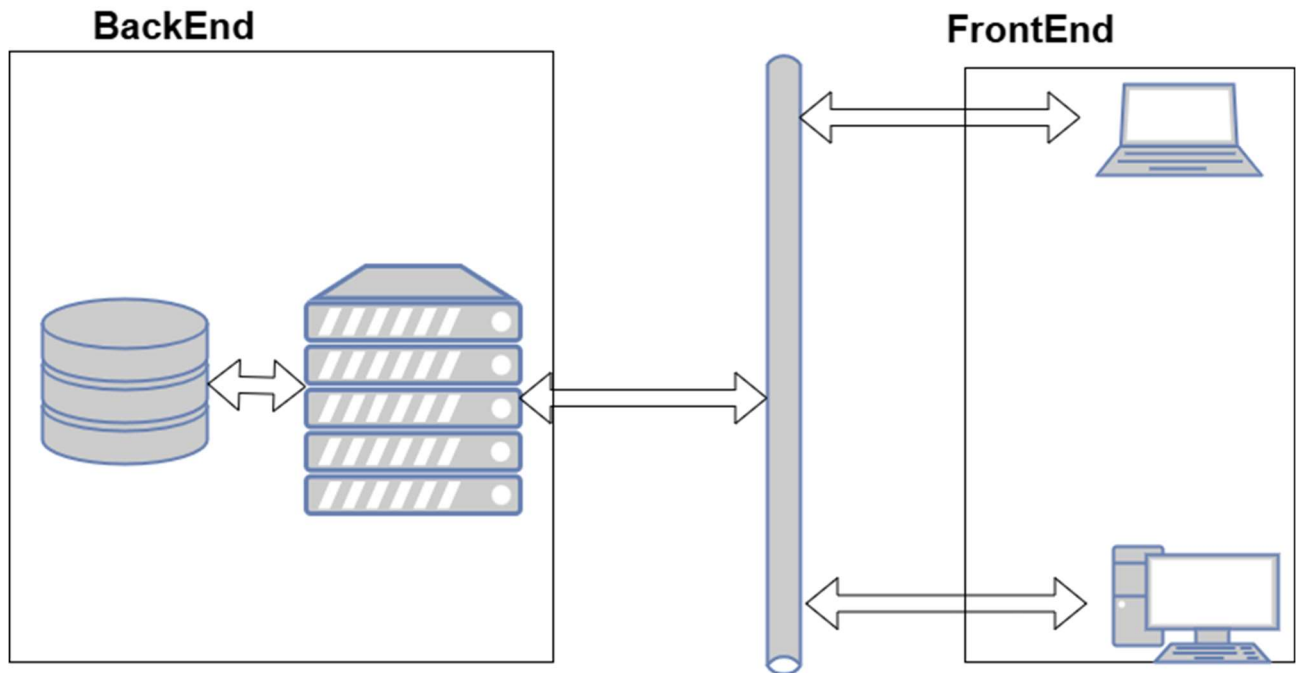
JavaScript, fetch, JSON, user session	- הצגת חברים - שליחת בקשת שיתוף עם מזהים	שליחה של פרויקט לחבר דרך תפריט שיתוף	שיתוף פרויקט עם חברים
JavaScript, fetch, socketio, דוא"ל	- שליחה עם אימות דוא"ל - קליטת תגובה והצגת אישור/שגיאה	שליחה למייל חבר דרך ממשק ייעודי	שליחת בקשת חברות
socketio, JavaScript, DOM	- חיבור ל- socket - האזנה לאירועים - הצגת popup	קבלת בקשות חברות והתראות WebSocket	קבלת התראות בזמן אמת
JavaScript, HTML, fetch, session	- דיאלוגים - שליחה לשרת והצגת שינויים	הוספה/עדכון/מחיקה של משימות	ניהול משימות

ארכיטקטורה של הפרויקט

○ תיאור הארכיטקטורה של המערכת המוצעת

המערכת המוצעת בנויה כאפליקציית ווב (Web Application) מבוססת Flask ב-Python המתבצעת באמצעות sockets. הארכיטקטורה מורכבת ממספר רכיבי שרת נפרדים, אשר מתפקדים יחד לבצוע פעולות מול בסיסי נתונים והצגת מידע למשתמש.

- Flask (צד שרת web): אחראי על קבלת בקשות מהמשתמש דרך הדפדפן, שליחתן לעיבוד פנימי והחזרת תגובה.
 - Server.py: מהווה שכבת תווך המקשרת בין Flask לבין רכיב ConnectionWithDatabase (צד המתווך בין Server.py לבין Network.py) הוא אחראי על תרגום הפקודות מצד ה-web לפעולות פנימיות שבסופן מעוברות דרך התקשורת.
 - שרת
 - שרת מסד נתונים: בהתאם לדרישות המערכת.
 - תחנת לקוח: דפדפן
- המשתמשים (FrontEnd) שולחים בקשות דרך הדפדפן. הבקשות עוברות לשרת (Backend). השרת מבצע עיבוד/שולף נתונים מה-Database. המידע נשלח חזרה למשתמשים לתצוגה.



BACKEND - תיאור מפורט

DATABASE

טבלאות נתונים שונות (מסומנות בצד שמאל): טבלת משתמשים (טיפול במשתמשים), טבלת פרויקטים (טיפול בפרויקטים), טבלת משימות (טיפול במשימות), טבלת חברים (טיפול בחברים), טבלת עובדים (טיפול בעובדים) **רכיב האחראי על שליפה והצגת מידע:** הצפה – שכבת ביניים שמטפלת בהכנה ושליחה של המידע הנדרש לשרת. **רכיב תקשורת:** מטפל בשליחת וקבלת נתונים לשרת דרך socket (תקשורת בזמן אמת).

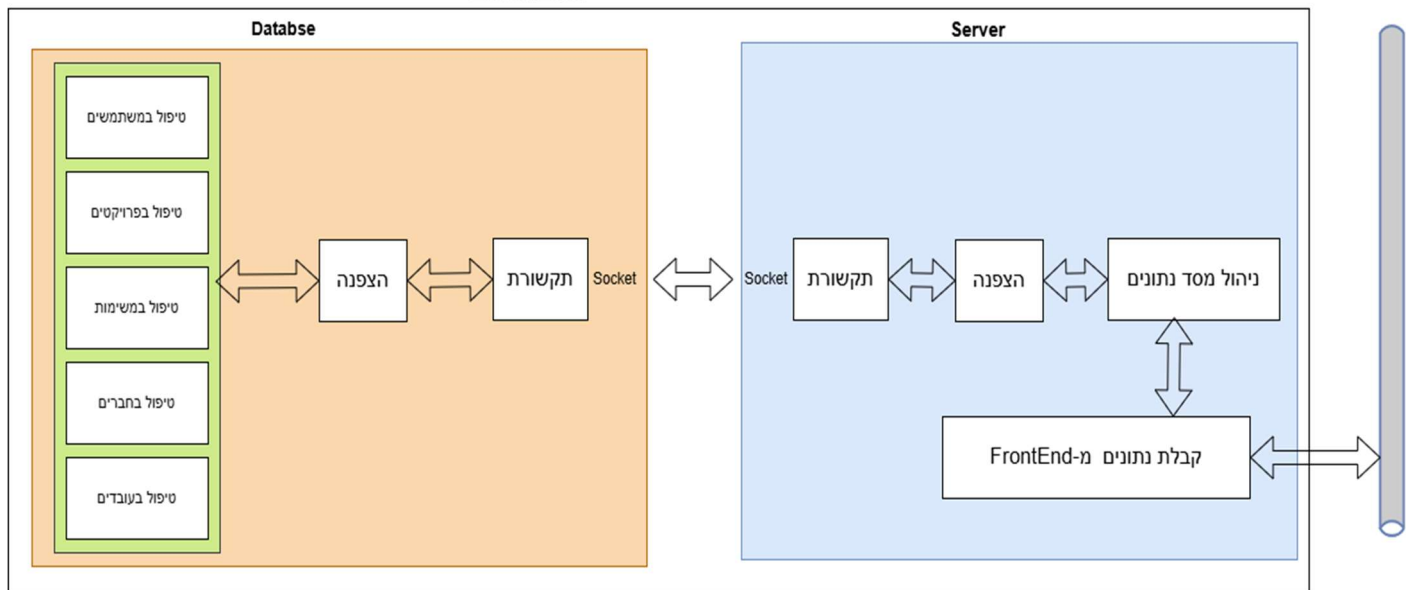
SERVER

ניהול מסד נתונים: אחראי על ניהול הנתונים שמתקבלים מה-Database עיבוד לוגי, בדיקות, סינונים ועוד. **קבלת נתונים מה-Frontend:** מקבל בקשות מהצד של המשתמשים (Frontend) ומעביר אותן הלאה לפי הצורך. **הצפה:** רכיב נוסף של הצפה, הפעם מהצד של השרת – משמש להכנת המידע שישלח למסד הנתונים. **תקשורת:** מטפל בתקשורת דרך Socket עם מסד הנתונים וגם עם ה-Frontend. מאפשר שליחה/קבלה של מידע בזמן אמת.

זרימת המידע:

משתמש שולח בקשה מה-Frontend (למשל: קבלת רשימת משימות).

הבקשה מגיעה לשרת → נשלחת דרך Socket לרכיב התקשורת. השרת מעבד את הבקשה ומעביר את הבקשה למנוע ההצפה. ההצפה שולחת את הבקשה למסד הנתונים. מסד הנתונים שולף את המידע (לפי סוג הנתונים: משתמשים, משימות וכו'). המידע עובר חזרה דרך ההצפה והתקשורת, חזרה לשרת. השרת מבצע עיבוד נוסף ומחזיר את התוצאה ל-Frontend.



FRONTEND - תיאור מפורט

כניסה (Login): מאפשר למשתמשים קיימים להזדהות עם שם משתמש וסיסמה. הבקשות נשלחות לשרת לצורך אימות.

הרשמה (Register): מיועד למשתמשים חדשים. מאפשר יצירת חשבון חדש במערכת. כולל הזנת פרטים בסיסיים ונשלח לשרת לשמירה במסד הנתונים.

ניהול חברים: ממשק לניהול רשימת חברים, שליחת בקשות חברות, קבלת בקשות וכו'.

ניהול ספרינטים (Sprints): מאפשר למשתמש לתכנן, להוסיף ולעקוב אחרי ספרינטים – קבוצת משימות בפרויקטים.

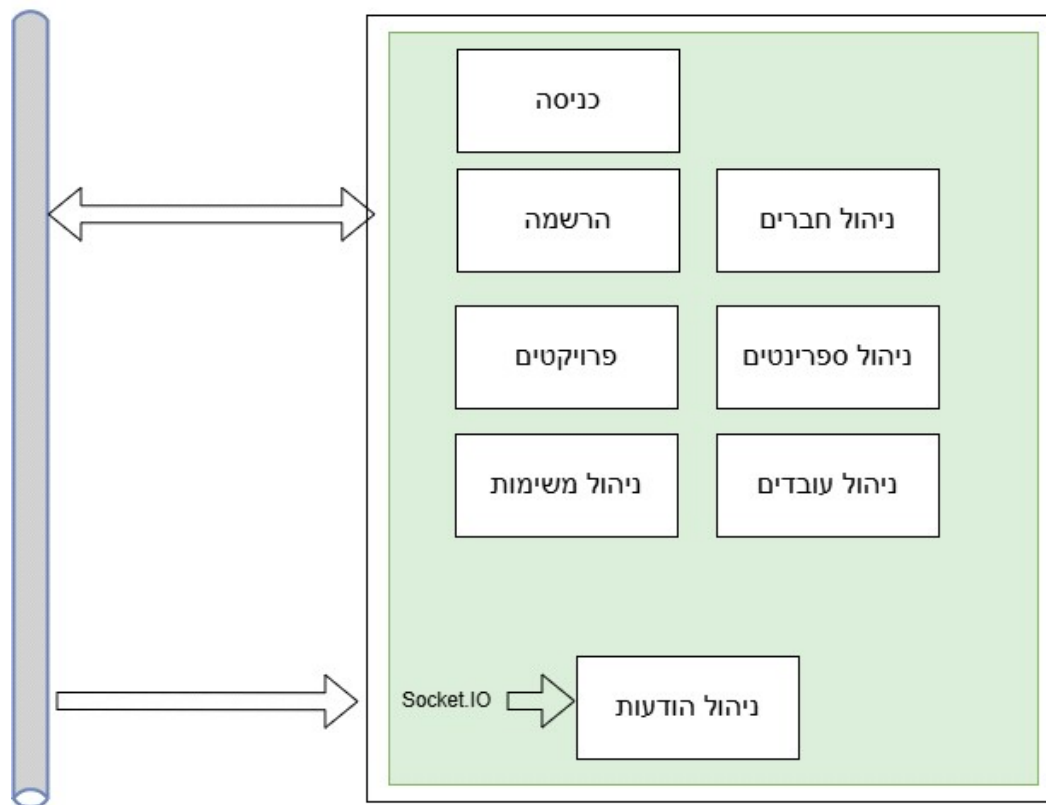
פרויקטים: כולל תצוגה ועריכה של פרויקטים שונים, הוספת פרויקטים חדשים, הקצאת משימות וכו'.

ניהול משימות: ניהול רשימת משימות תחת פרויקטים או ספרינטים. כולל סטטוסים, עדיפויות, תאריכים ועוד.

ניהול עובדים: מיועד למנהלים – לניהול כוח אדם, תפקידים, הרשאות וכו'.

ניהול הודעות (Socket.IO): מודול מיוחד שמשתמש בטכנולוגיית Socket.IO לצורך קבלת ושליחת הודעות בזמן אמת (למשל: התראות, בקשות חברות, עדכונים מערכתיים).

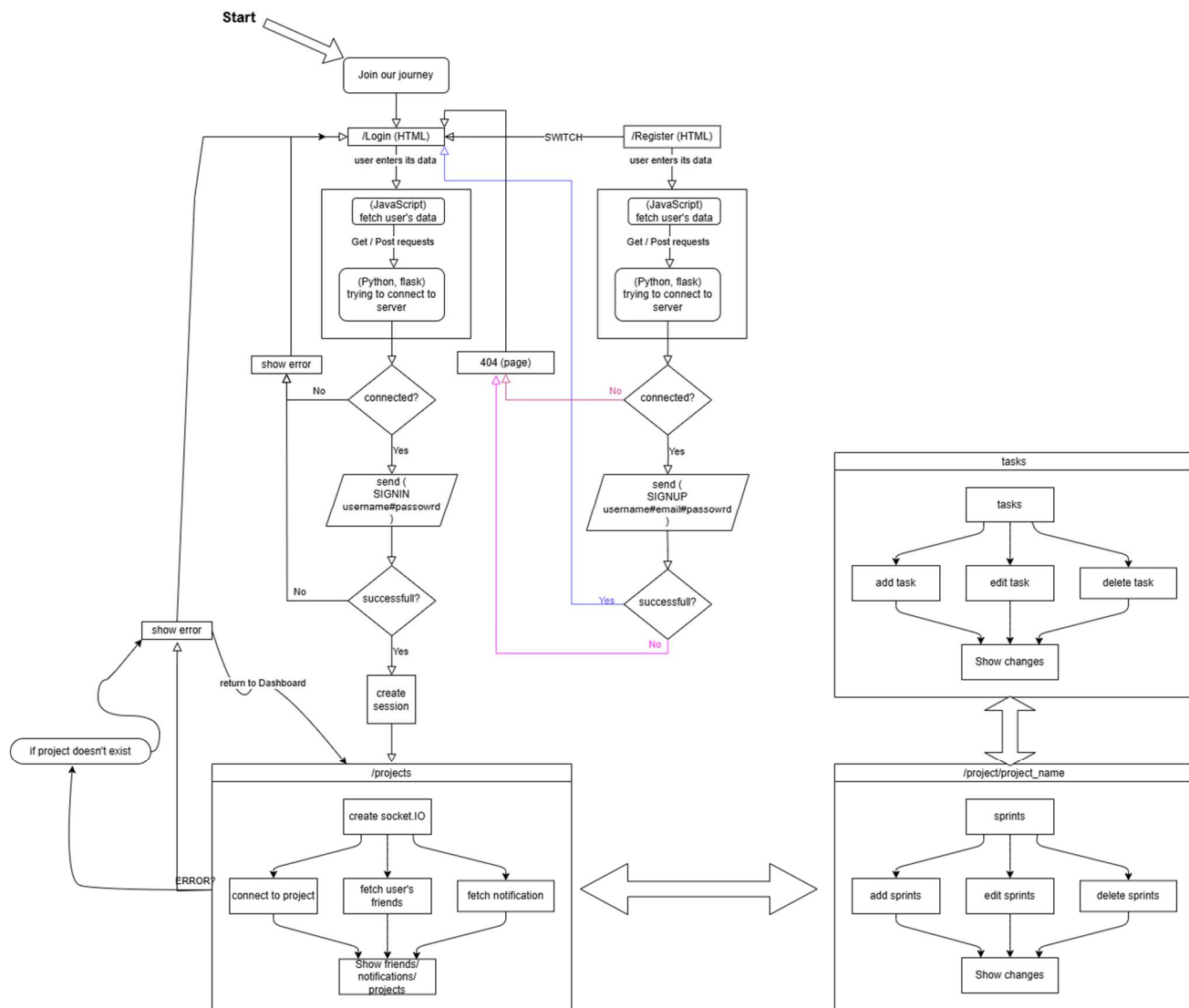
FrontEnd



○ תיאור הטכנולוגיה הרלוונטית

בצד שרת – אני משתמש ב – Flask, Socket, Socket.IO

בצד לקוח – אני משתמש ב – HTML, JavaScript, CSS, WebSocket



תיאור האלגוריתמים מרכזים בפרויקט

כדי לאבטח את התעבורה השתמשתי בהצפנה מסוג RSA.

RSA מבוססת על עקרונות מתמטיים של תורת המספרים, ובפרט על הקושי בפירוק של מספרים שלמים גדולים למכפלה של ראשוניים. בהצפנת RSA נוצרים זוג מפתחות: מפתח ציבורי ומפתח פרטי. תהליך יצירת המפתחות כולל את השלבים הבאים:

1. בחירה של שני מספרים ראשוניים גדולים, p ו- q
2. חישוב המכפלה שלהם: $q * p = n$ המספר n משמש כחלק מהמפתח הציבורי והפרטי.
3. חישוב של פונקציית אוילר: $\phi(n) = (1 - q)(1 - p)$
4. בוחרים מספר שלם בשם e , כך שהוא גדול מ-1 וקטן מ- $\phi(n)$, ואין שאין לו מחלקים משותפים עם $\phi(n)$ מלבד 1 – כלומר שהוא זר ל- $\phi(n)$ (עוקב לו). המספר הזה ישמש כ"חשיפת" המפתח הציבורי (האקספוננט הציבורי) בהצפנת RSA.
5. כדי שהצפנת RSA תעבוד, אנחנו צריכים שני מספרים: אחד להצפנה (ציבורי) ואחד לפענוח (סודי). את המספר להצפנה בחרנו קודם – זה המספר e . עכשיו אנחנו צריכים למצוא מספר אחר – נקרא לו d – שהוא "משלים" של e בצורה כזו, שכאשר נשתמש בו על הודעה שהוצפנה עם e , נקבל בחזרה את ההודעה המקורית.

6. בשפה פשוטה, אנחנו מחפשים מספר d ש"מאזן" את e – כמו הפוך מכפל, אבל לא בעולם הרגיל, אלא בעולם של שאריות לפי מספר מסוים שנקרא $\phi(n)$ שנובע מהמספרים הראשוניים שבחרנו בהתחלה).

הבעיה האלגוריתמית המרכזית ניהול פרויקטים בו-זמנית. הבעיה מתחלקת לתת-בעיות הבאות:

1. התמודדות עם ריבוי-משתמשים
2. ניהול משימות בתוך פרויקט : הוספת משימה, מחיקת משימה, עדכון משימה.

○ תיאור אלגוריתמים קיימים לפתרון הבעיה

בכדי להתמודד עם ריבוי-משתמשים השתמשתי מאחסן מצב של משתמש (STATE) ב-session. session -ה מאחסן:

session_id – משמש כמעין מפתח אישי להבדיל בחיבור תקשורת עם מסד הנתונים עם כניסה של כל משתמש. השתמשתי במודול uuid על מנת ליצור מזהה רנדומלי לאובייקט התקשורת ובכך להבדיל בין משתמשים.

ניהול משימות:

הוספת משימה - תהליך הוספת משימה מתבצע על ידי קריאת POST מהצד הקדמי, אשר מעבירה את מזהה הספרינט (SprintID) ואת שם משימה (TaskName). תחילה, המערכת מקבלת את שם המשתמש דרך ה-Session. לאחר מכן, האלגוריתם סורק המשימות של המשתמש (session["tasks"]) באמצעות לולאה את כל הספרינטים הקיימים בזיכרון המשתמש (session["sprints"]) ומחפש את הספרינט עם אותו מזהה. אם נמצא הספרינט - האלגוריתם שומר את שם הספרינט (sprint_name) באופן זמני ויוצר משימה חדשה המכילה את המזהה של המשימה, שם המשימה, נתוני המשימה (כגון: סטטוס, עדיפות, התקדמות ועוד). אחרי שהמשימה מוכנה, היא נשלחת בזמן אמת לכל שאר המשתמשים שמחוברים לאותו פרויקט, בעזרת Socket.IO, וגם נשלחת לשרת הראשי כדי שהשינויים יישמרו באופן קבוע במסד הנתונים.

מחיקת משימה - תהליך מחיקת משימה מתבצע על ידי קריאת POST מהצד הקדמי, אשר מעבירה את מזהה המשימה (TaskID) ואת מזהה הפרויקט (ProjectID). תחילה, המערכת מקבלת את שם המשתמש דרך ה-Session ומאחסנת את מזהי המשימה והפרויקט ב-Session. לאחר מכן, האלגוריתם סורק באמצעות לולאה את כל הספרינטים הקיימים בזיכרון המשתמש (session["sprints"]) ומחפש את המשימה עם אותו מזהה. אם נמצאה - המשימה נמחקת (תוסר) מרשימת המשימות של אותו הספרינט. האלגוריתם שולח את הוראה לשרת מסד הנתונים למחוק את המשימה בכדי לשמור את השינויים באופן קבוע. על מנת לעדכן את שאר המשתמשים בפרויקט, המערכת שולחת בקשת מחיקה דרך החיבור לשרת ולאחר קבלת אישור שהמחיקה בוצעה בהצלחה - משודרת התראה לכל המשתמשים שמחוברים לחדר הפרויקט באמצעות Socket.IO. בנוסף ההתראה כוללת את מזהה הפרויקט, מזהה המשימה שנמחקה, שם הספרינט ממנו הוסר, ושם המשתמש שביצע את המחיקה. אם המחיקה לא צלחה, תחזיר המערכת הודעת שגיאה האומרת כי לא נמצאה המשימה.

עדכון משימה - תהליך עדכון המשימה מתחילה בקליטת הנתונים החדשים של המשימה (כגון שם חדש, Status חדש, Priority חדש וכו') דרך קריאת POST. באמצעות לולאה, האלגוריתם סורק את כל הספרינטים (Sprints) שנשמרו אצל המשתמש ב-Session. בתוך כל ספרינט הוא מחפש את המשימה עם אותו מזהה (ID). ברגע שמוצא - שומר אותו לעדכון. האלגוריתם משווה בין הערכים הנוכחיים של המשימה לבין מה שנשלח מהמשתמש. אם יש שינוי בשם, סטטוס או עדיפות - הוא מעדכן את הערכים ושומר רשימת שינויים. בנוסף האלגוריתם מורה למערכת לשדר את המשימה המעודכנת (יחד עם רשימת השינויים) דרך Socket.IO לכל המשתמשים שעובדים על אותו הפרויקט בזמן אמת, בכך המשתמשים מודעים לשינויים שנעשו על ידי המשתמש. המערכת דואגת שגם ב-Session (בצד של המשתמש שביצע את שינוי) תתאחסן ותתעדכן המשימה החדשה. לבסוף, האלגוריתם שולח את המשימה המעודכנת גם לשרת מסד הנתונים כדי לשמור את השינויים באופן קבוע.

PyCharm 2024.1.2 (Community Edition) Runtime version: 17.0.11+1-b1207.24 amd64

DB Browser for SQLite Version 3.13.1 Windows 10.0

צד שרת: שפות תכנות Python 3.12

גישה למסד נתונים ע"י SQL

צד לקוח: עיצוב HTML, CSS

שפות תכנות JavaScript

○ תיאור פרוטוקול התקשורת

פרוטוקול החלפת מפתחות

השרת מתחבר לשרת מסד הנתונים. שרת מסד הנתונים שולח לשרת RSA public key. השרת שולח (לשרת) מסד הנתונים key AES שמוצפן על ידי ה- public RSA key. שרת מסד הנתונים מפענח את המפתח ששלח לו השרת באמצעות ה- RSA private key. גם שרת מסד הנתונים וגם השרת יוצרים אובייקט AES בו נשמר המפתח מסוג AES. (ראו תרשים)

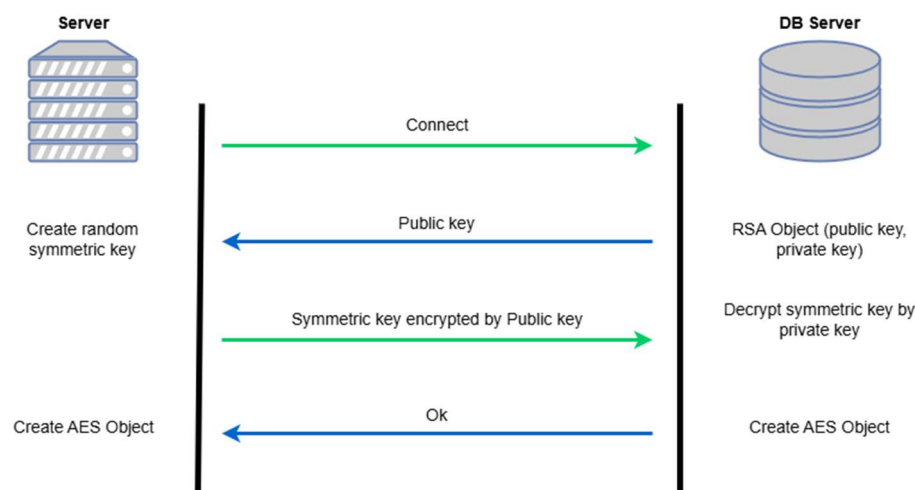
מפתח ציבורי: המפתח הציבורי הנשלח מהשרת ללקוח מוצפן בפורמט המתאים לרכיב RSA עם גודל מפתח של 1024 ביטים.

פירוט כלל ההודעות הזורמות במערכת

הודעה: שליחת מפתח ציבורי:

- שם ההודעה:
- נשלחת מ/אל: משרת מסד הנתונים אל שרת
- מבנה השדות בהודעה:

○ מפתח ציבורי: אורך משתנה, בפורמט המתאים להצפנת RSA עם גודל מפתח של 1024 ביטים.



פרוטוקול בין שרת לבין שרת מסד נתונים

מוגדר לתקשורת בין שרת מסד הנתונים (שרת) לבין השרת (לקוח) על ידי שימוש בחיבור IP/TCP עם מפתח הצפנה פרטי אישי לכל חיבור שמקיים הלקוח עם השרת.

תשובה משרת מסד הנתונים לשרת: שרת מסד הנתונים שולח לשרת תשובה בפורמט טקסטואלי, הכוללת את המידע שהתבקש. מבנה ואורך התשובה משתנים בהתאם לסוג הבקשה ולתוכן הנתונים שנשלפו.

פירוט כלל ההודעות הזורמות במערכת

הודעה: שליחת מפתח ציבורי:

- שם ההודעה:
- נשלחת מ: שרת מסד הנתונים
- נשלחת אל: שרת
- מבנה השדות בהודעה:
- מפתח ציבורי: אורך משתנה, בפורמט המתאים להצפנת RSA עם גודל מפתח של 1024 ביטים.

הודעה: התחברות - שליחת שם משתמש וסיסמה

- שם ההודעה: **SIGNIN**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **SIGNIN** username#password
- שם משתמש (**username**): מחרוזת המכילה את שם המשתמש. סיסמה (**password**): מחרוזת המכילה את הסיסמה שהזין המשתמש.

הודעה: הרשמה - שליחת שם משתמש, אימייל וסיסמה

- שם ההודעה: **SIGNUP**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **SIGNUP** username#email#password
- שם משתמש (**username**): מחרוזת המכילה את שם המשתמש. אימייל (**email**): מחרוזת שתומך מחרוזת נמוכה (lower, אותיות קטנות) בהמשך. סיסמה (**password**): מחרוזת המכילה את הסיסמה שהזין המשתמש.

הודעה: הוספת בקשת חברות - שליחת מזהה של המשתמש שולח ומקבל

- שם ההודעה: **ADD_FRIEND_REQUEST**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **ADD_FRIEND_REQUEST**#sender_id.receiver_id
- מזהה של המשתמש (**sender_id**): int המכיל את מזהה של המשתמש ששלח את הבקשה.
- מזהה של המשתמש (**receiver_id**): int המכיל את מזהה של המשתמש שמקבל את הבקשה.

הודעה: השג את האימייל של משתמש - שליחת מזהה של המשתמש

- שם ההודעה: **GET_USER_EMAIL**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **ADD_FRIEND_REQUEST**#receiver_id
- מזהה של המשתמש (**receiver_id**): int המכיל את מזהה של המשתמש שדרכו נקבל את האימייל שלו.
-

הודעה: השג את המזהה של משתמש בעזרת אימייל - שליחת אימייל של המשתמש

- שם ההודעה: **GET_USER_BY_EMAIL**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **GET_USER_BY_EMAIL**#email
- אימייל (**email**): string המכילה את אימייל של המשתמש שדרכו נקבל את המזהה שלו.

הודעה: השג את הבקשות הממתינות - שליחת אימייל משתמש

- שם ההודעה: **GET_PENDING_REQUESTS**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **GET_PENDING_REQUESTS**#email
- אימייל (**email**): string המכילה את אימייל של המשתמש שדרכו נקבל את הבקשות הממתינות שלו.

הודעה: אישור חברות - שליחת מזהה של המשתמש שולח ומקבל

- שם ההודעה: **CONFIRM_FRIENDSHIP**
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- **CONFIRM_FRIENDSHIP**#sender_id.receiver_id

- מזהה של המשתמש (sender_id): int המכיל את מזהה של המשתמש ששלח את הבקשה.
- מזהה של המשתמש (receiver_id): int המכיל את מזהה של המשתמש שמאשר את הבקשה.
- הודעה: השג את החברים – payload of data
 - שם ההודעה: CONFIRM_FRIENDSHIP
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - GET_FRIENDS#payload
 - payload : json.dumps של dictionary ← רשימה שבתוכה מילון.
 - המילון בעל:
 - מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שעליו מדובר.
 - משתמשים "אקטיביים" (פעילים - מחוברים, active_users): list של מזהה (int) המכיל את חברי המשתמש (client_id)
- הודעה: השג את העובדים – מזהה של משתמש בעל סמכות מנהל
 - שם ההודעה: GET_WORKERS
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - GET_WORKERS#manager_id
 - מזהה של המשתמש (manager_id): int המכיל את מזהה של המשתמש בעל סמכות מנהל.
- הודעה: שלח בקשת עובד – מזהה של משתמש בעל סמכות מנהל, אימייל של העובד
 - שם ההודעה: SEND_REQUEST_WORKER
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - SEND_REQUEST_WORKER#manager_id.worker_email
 - מזהה של המשתמש (manager_id): int המכיל את מזהה של המשתמש בעל סמכות מנהל.
 - אימייל (worker_email): string המכילה את אימייל של העובד אותו המנהל מבקש להעסיק
- הודעה: שלח בקשת עובד – מזהה של משתמש בעל סמכות מנהל, אימייל של עובד
 - שם ההודעה: ADD_WORKER
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - ADD_WORKER#manager_id.worker_email
 - מזהה של המשתמש (manager_id): int המכיל את מזהה של המשתמש בעל סמכות מנהל.
 - אימייל (worker_email): string המכילה את אימייל של עובד אותו המנהל ליקט.
- הודעה: תן פרויקטים – מזהה של משתמש
 - שם ההודעה: GIVE_PROJECTS
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - GIVE_PROJECTS#client_id
 - מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שבעבורו יקבל את הפרויקטים הישירים.
- הודעה: הוסף פרויקט – מזהה של משתמש
 - שם ההודעה: ADD_ITEM
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - ADD_ITEM#client_id
 - מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שבעבורו יתווסף פרויקט חדש.
- הודעה: שכפל פרויקט – מזהה של משתמש ושל פרויקט
 - שם ההודעה: DUPLICATE_ITEM
 - נשלחת מ: שרת
 - נשלחת אל: שרת מסד הנתונים
 - מבנה השדות בהודעה:
 - DUPLICATE_ITEM#client_id.project_id
 - מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שבעבורו ישוכפל פרויקט זה.

- מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט המיועד לשיכפול.
- הודעה: שתף פרויקט או הקצה פרויקט – מזהה של שני משתמשים ושל פרויקטים
- שם ההודעה: SHARE_PROJECT
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- SHARE_PROJECT#owner_id.client_id.project_id
- מזהה של המשתמש (owner_id): int המכיל את מזהה של המשתמש בעל אותו פרויקט שישותף.
- מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שאיתו ישותף הפרויקט.
- מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט המיועד לשינוף.
- הודעה: עדכן – תלוי בנושא אותו רוצים להעדכן
- שם ההודעה: 'UPDATE'
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- UPDATE subject\$client_id.subject_id#updated_data
- נושא לעדכון (subject): string המכילה את הנושא שאותו מעוניינים לעדכן (פרויקטים, משימות)
- מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש המיועד לעדכון הנתונים.
- מזהה של פרויקט/משימה (subject_id): int המכיל את מזהה של הפרויקט/משימה התעיד על שפרויקט או משימה זו תעודכן.
- נתונים מעודכנים (update_data): list (המכילה רק מילון אחד וזהו). בתוך המילון נמצאים הנתונים אותם צריך להעדכן.
- הודעה: מחק – תלוי בנושא אותו רוצים למחוק/להסיר
- שם ההודעה: 'DELETE'
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- DELETE subject#client_id.project_id
- נושא לעדכון (subject): string המכילה את הנושא שאותו מעוניינים להסיר (פרויקטים, משימות)
- מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש המיועד למחיקת הנתונים.
- מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט שממנו נמחק את פרויקט או נסיר את המשימה ממנו.
- הודעה: תן – תלוי בנושא אותו רוצים לקבל
- שם ההודעה: 'GIVE'
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- GIVE client_id.project_id#subject
- נושא לעדכון (subject): string המכילה את הנושא שאותו מעוניינים לקבל (ספרינטים, תגובות)
- מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שממנו ניקח את הנתונים.
- מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט שממנו נקבל את הספרינטים/תגובות שנמצאים תחתיו.
- הודעה: עדכן ספרינט – מזהה ספרינט ונתונים לעדכון
- שם ההודעה: UPDATE_SPRINT
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- UPDATE_SPRINT#sprint_id\$data
- נתונים לעדכון (data): dict המכילה את נתונים שצריך לעדכן. (sprint_name, sprint_color)
- מזהה של ספרינט (sprint_id): int המכיל את מזהה של הספרינט אותו נרצה לעדכן את הנתונים שלו.
- הודעה: הוסף ספרינט – מזהה של משתמש, מזהה של פרויקט ונתוני הספרינט החדש
- שם ההודעה: ADD_SPRINT
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:
- ADD_SPRINT#client_id.project_id\$data
- נתונים על ספרינט החדש (data): dict המכילה את נתונים שצריך להוסיף. (sprint_id, sprint_name, sprint_color)
- מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שנוסף תחתיו את הנתונים החדשים.

○ מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט שיכונן אותנו לאיזה פרויקט רוצים את הספרינט החדש.

הודעה: מחק ספרינט – מזהה פרויקט, מזהה ספרינט

- שם ההודעה: DEL_SPRINT
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:

○ DEL_SPRINT#project_id.sprint_id

○ מזהה של ספרינט (sprint_id): int המכיל את מזהה של הספרינט המיועד למחיקה.

○ מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט שממנו נסיר את הספרינט.

הודעה: עדכן משימה – תלוי בנושא אותו רוצים למחוק/להסיר

- שם ההודעה: UPDATE_TASK
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:

○ UPDATE_TASK\$data

○ נתונים על משימה (data): dict המכילה את נתונים שצריך לעדכן:

(ID, TaskName, Priority, Status, status_color, priority_color, Owners, DueDate, Tags, Progress)

הודעה: הוסף משימה – מזהה של משתמש, מזהה של פרויקט, מזהה של ספרינט, ושם המשימה החדשה

- שם ההודעה: ADD_TASK
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:

○ ADD_TASK#client_id.project_id.sprint_id.task_id\$task_name

○ מזהה של המשתמש (client_id): int המכיל את מזהה של המשתמש שנוסף תחתיו את הנתונים החדשים.

○ מזהה של פרויקט (project_id): int המכיל את מזהה של הפרויקט שיכונן אותנו לאיזה פרויקט רוצים את

הספרינט החדש.

○ מזהה של ספרינט (sprint_id): int המכיל את מזהה של הספרינט שיכונן אותנו לאיזה ספרינט רוצים את

המשימה החדשה.

○ מזהה של משימה (task_id): int המכיל את מזהה של המשימה שיכונן אותנו את המשימה החדשה שרוצים

להוסיף.

○ שם המשימה (task_name): string המכיל את שם המשימה החדשה שתווסף.

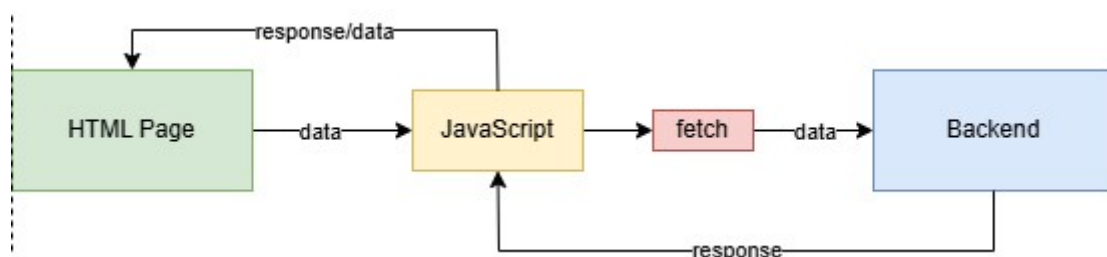
הודעה: סיים

- שם ההודעה: END
- נשלחת מ: שרת
- נשלחת אל: שרת מסד הנתונים
- מבנה השדות בהודעה:

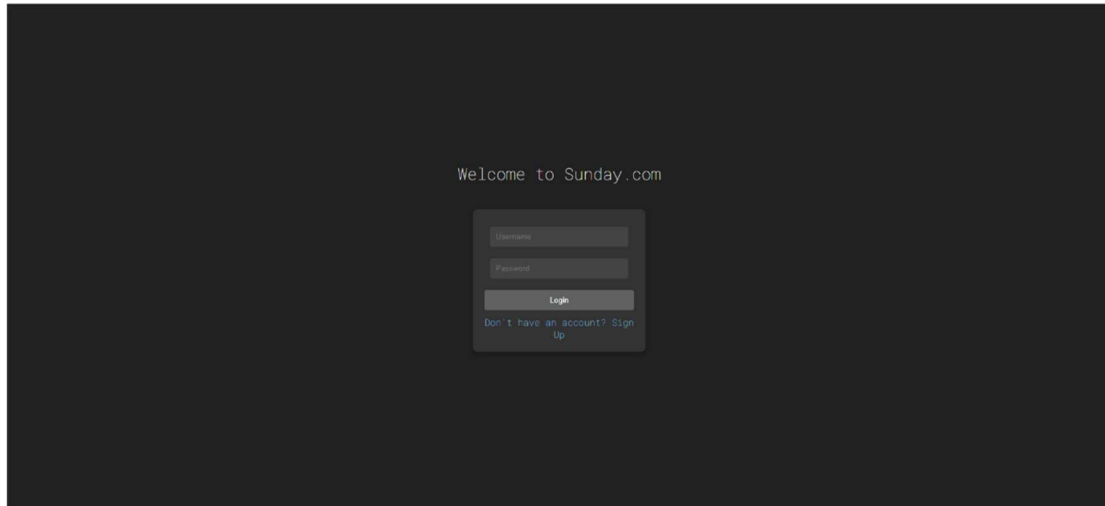
○ string : מעידה על סיום התקשורת עם משתמש. END

פרוטוקול בין שרת (BACKEND) לבין דפדפן (FRONTEND)

על מנת שנוכל ללקט את המידע שהמשתמש מזין בדפדפן אל תוך ה-Backend, נעזר ב-HTML ו-JavaScript. תחילה בצד ה-Frontend יוצרים טופס HTML שבו המשתמש מזין את המידע. המשתמש מזין מידע. JavaScript שולח את המידע לשרת באמצעות fetch. צד השרת (Backend) מקבל את הבקשה ומחזיר תשובה בהתאם לצד ה-Frontend באמצעות JSON דרך HTML.



מסך הכניסה

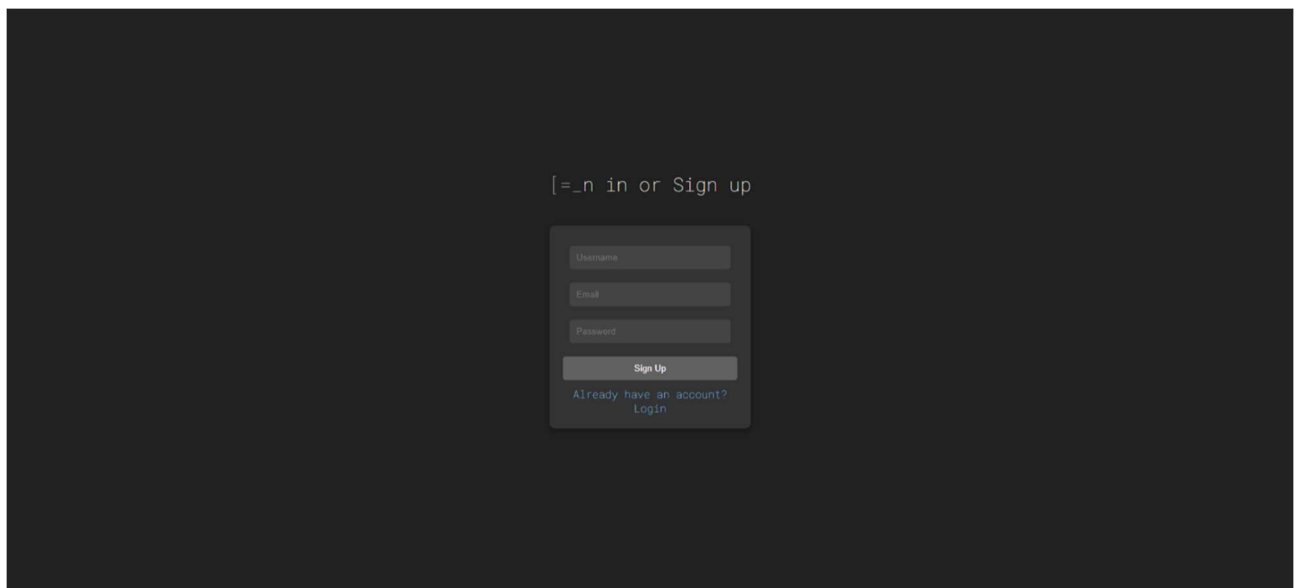


מסך הכניסה מאפשר למשתמש קיים להיכנס/לחזור למסך הבית ולהתחיל לעבוד.

מה ניתן לבצע בעמוד הזה

- לרשום את שם המשתמש הקיים, הסיסמה שלו, וללחוץ Enter בסיום קליטת הנתונים.
- אם אין משתמש קיים, יש ללחוץ על ההרשמה המופיע מתחת ל- login בכחול.

מסך ההרשמה



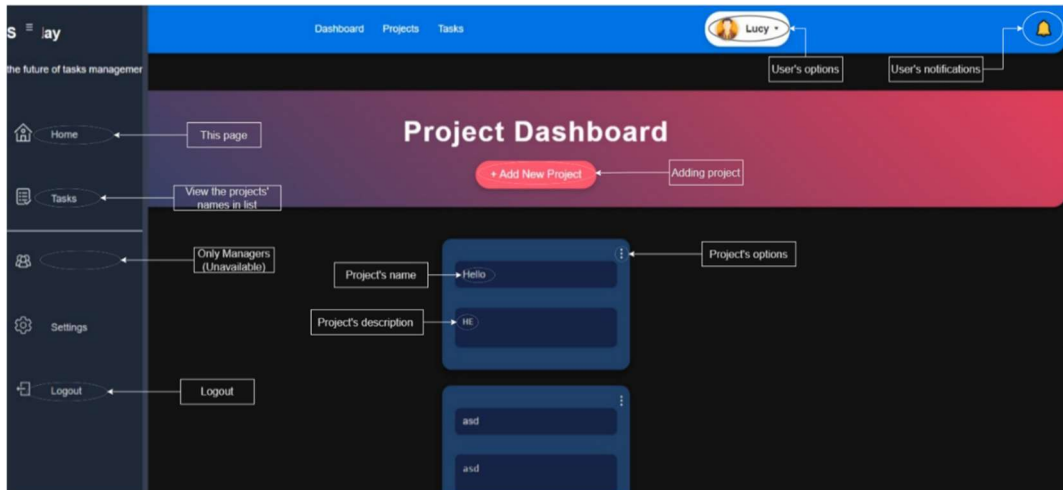
מסך ההרשמה מאפשר למשתמש חדש להירשם למערכת.

מה ניתן לבצע בעמוד הזה

• לרשום את שם המשתמש, האימייל, הסיסמה שלו, וללחוץ Enter בסיום קליטת הנתונים.

• אם יש משתמש קיים, יש ללחוץ על ההרשמה המופיע מתחת ל- sign up בכחול.

מסך הבית - פרויקטים

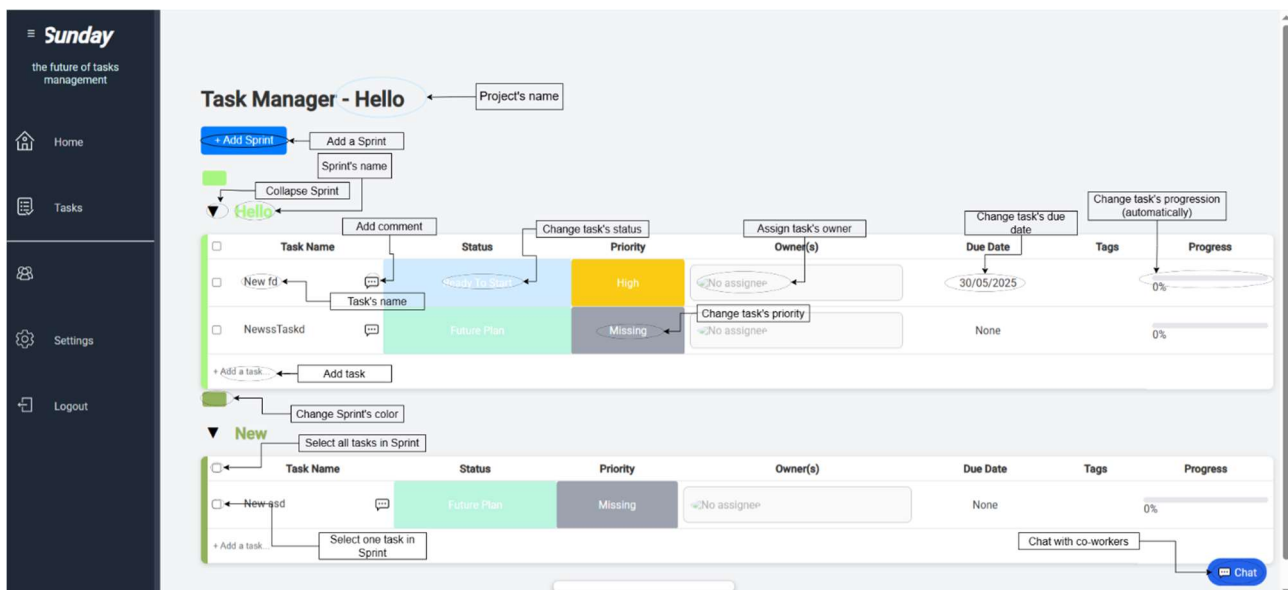


מסך הבית מציג את הפרויקטים שיצר לעצמו המשתמש, וגם מציג את הפרויקטים המשותפים עם חבריו. בנוסף אם ייצרף המשתמש חברים, תוצג רשימת החברים מעל כפתור "Add project". העמוד מכיל גם ניווט צדדי המאפשר: לחזור לעמוד הראשי (הנוכחי) – פרויקטים. הגדרות כלליות למשתמש. התנתקות מהמערכת.

מה ניתן לבצע בעמוד הזה

- צפייה ברשימת הפרויקטים הקיימים.
- הוספת פרויקט חדש בקלות.
- ניהול פרויקטים קיימים (שם, תיאור) ושיתוף הפרויקטים עם חברים ו/או עובדים.

מסך המשימות

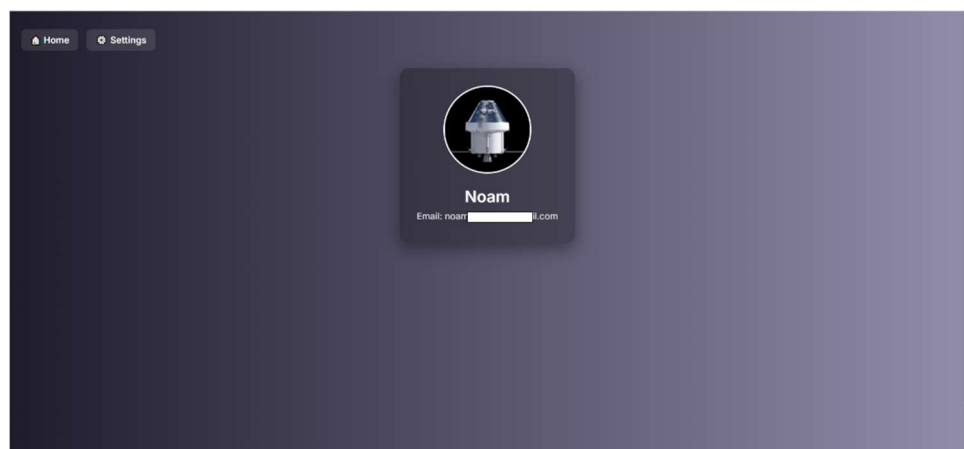


מסך המשימות מציג את המשימות שיצר לעצמו המשתמש. המשתמש יכול להוסיף sprint (קבוצת משימות). העמוד מכיל גם ניווט צדדי

מה ניתן לבצע בעמוד הזה

- צפייה במשימות הקיימות בפרויקט שנפתח.
- הוספת משימות וקבוצת משימות (sprint) חדשה בקלות.
- קביעת נתונים שונים, כגון: שם המשימה, סטטוס, עדיפות, תאריך יעד, שינוי שם וצבע של קבוצת המשימות.
- כשחברים נוספו לפרויקט וצופים באותו העמוד, יוכלו החברים לעקוב אחר שינויים שנעשו בזמן אמת

מסך המשתמש

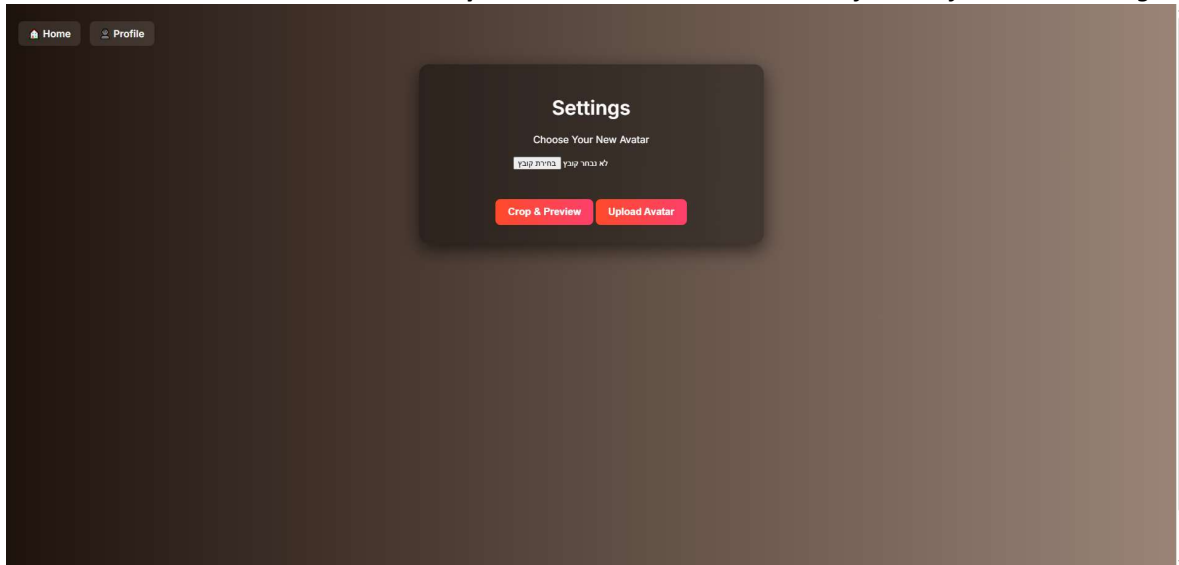


מסך המשתמש (profile) מציג את שם המשתמש, יחד עם האימייל הזין לעצמו המשתמש בהרשמה.

מה ניתן לבצע בעמוד הזה

- לחזור לעמוד הראשי
- ללחוץ על ההגדרות במסך למעלה.

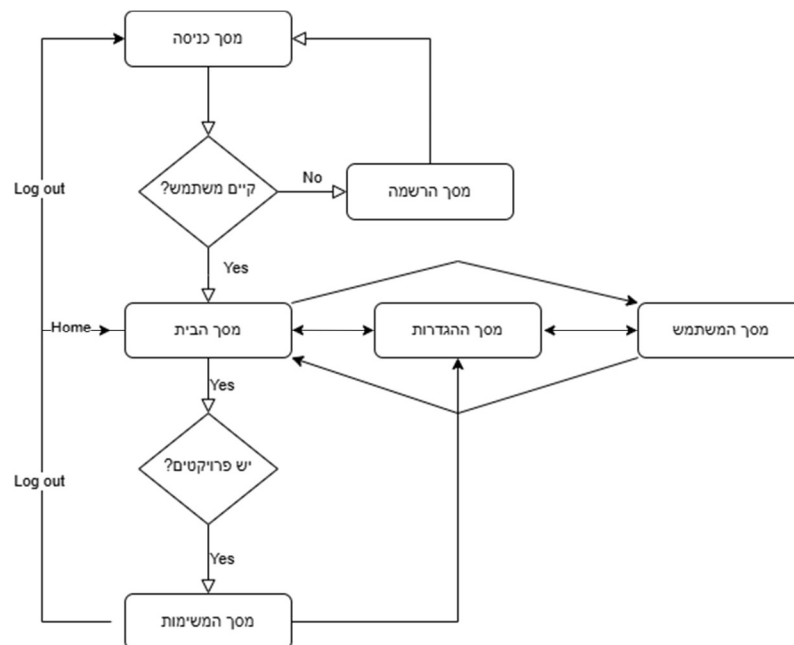
מסך ההגדרות

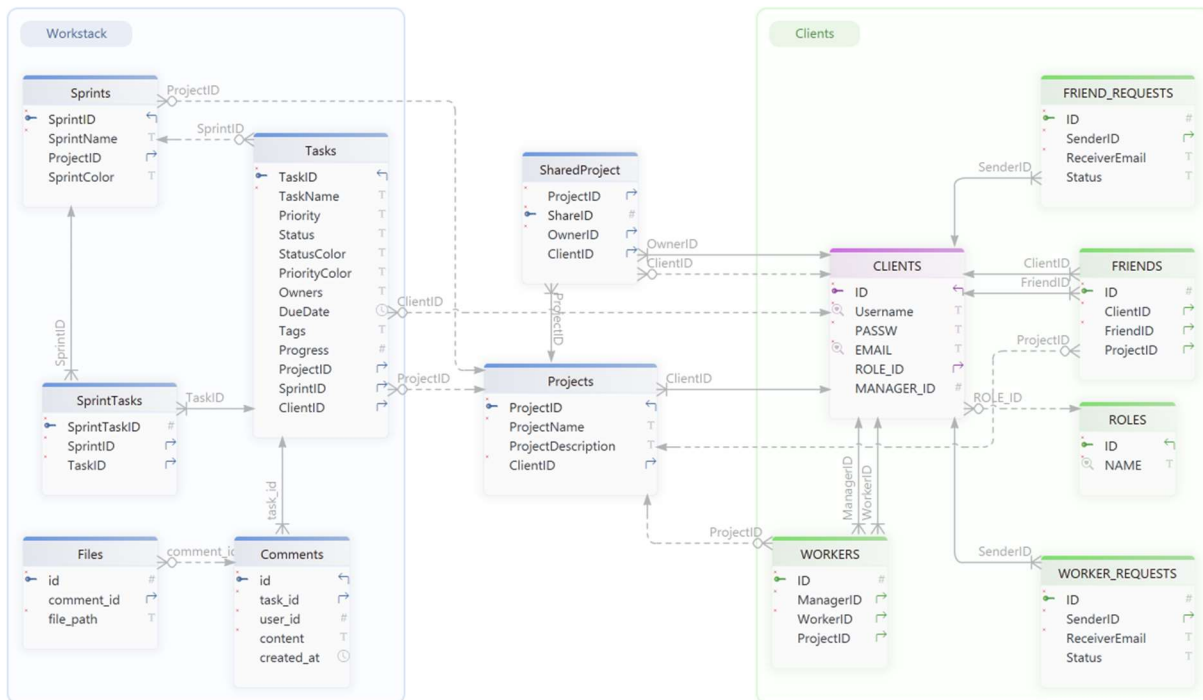


מסך ההגדרות (settings).

מה ניתן לבצע בעמוד הזה

- לחזור לעמוד הראשי
- לשנות את תמונת הפרופיל באופן אישי, כלומר לבחור תמונה מהמחשב, לערוך את הגודל ולשמור את השינויים. לאחר השינוי, תעודכן התמונה וכל חברי המשתמש יראו את השינוי (refresh)





החצים העוברים בין הטבלאות הם ה- *foreign_keys* של הטבלאות

CLIENTS (בסגול) – מאפשר אחסון של משתמשים. ביניהם המזהה האישי של כל אחד, שם המשתמש, האימייל שלו, סוג התפקיד ואם יש מנהל, אז המזהה של המנהל.

ROLES (בירוק, משמאל ל-CLIENTS) – מאחסן סוג התפקיד. (דוגמה, אם המשתמש הוא עובד, אז ה-ROLE_ID בטבלה של CLIENTS יהיה 2)

Projects – מאפשר אחסון של פרויקטים. ביניהם מזהה הפרויקט, שם הפרויקט, מזהה המשתמש את הפרויקט, בעל הפרויקט (מזהה של משתמש מטבלת ה-CLIENTS).

ShareProjects – מאפשר אזכור של פרויקטים משותפים. ביניהם מזהה הפרויקט המשותף, בעל הפרויקט ומזהה המשתמש איתו הפרויקט משותף (2 המזהים של משתמשים הם מטבלת ה-CLIENTS).

Workers – מאפשר אזכור של עובדים. ביניהם המזהה של המנהל, מזהה של עובד, ומזהה של הפרויקט שעליו הם עובדים (מזהה של פרויקט מטבלת ה-Projects).

WORKER_REQUESTS – מאפשר אחסון של בקשות עובדים. ביניהם מזהה של שולח הבקשה, אימייל של המשתמש שאמור לקבל את הבקשה ואת הסטטוס של הבקשה (ממתין, נדחתה). אם אושר, אז תוסר הבקשה ממסד הנתונים.

FRIEND_REQUESTS – מאפשר אחסון של בקשת חברות. ביניהם מזהה של שולח הבקשה, אימייל של המשתמש שאמור לקבל את הבקשה ואת הסטטוס של הבקשה (ממתין, נדחתה). אם אושר, אז תוסר הבקשה ממסד הנתונים.

FRIENDS – מאפשר אזכור של חברים. ביניהם המזהה של המשתמש, מזהה של החבר, ומזהה של הפרויקט שעליו הם יכולים לעבוד ביחד (מזהה של פרויקט מטבלת ה-Projects, 2 המזהים של משתמשים הם מטבלת ה-CLIENTS).

TASKS – מאפשר אחסון של משימות. ביניהם המזהה של המשימה, שם המשימה, דחיפות המשימה, סטטוס המשימה (צבעי הסטטוסים ודחיפות), בעלי המשימות (אלה שלוקחים אחריות/קיבלו את ההוראה לבצע את המשימה), תאריך הגשת/סיום המשימה, התקדמות המשימה באחוזים, מזהה הפרויקט (איפה הוא נמצא), מזהה של הספרינט (איפה בדיוק הוא נמצא), ומזהה של המשתמש. כלומר, למי זה שייך. (מזהה של פרויקט מטבלת ה-Projects, 2 המזהים של משתמשים הם מטבלת ה-CLIENTS וה-SPRINTS).

SPRINTS – מאפשר אחסון של ספרינטים. ביניהם המזהה של הספרינט, שם הספרינט, צבע הספרינט, מזהה הפרויקט (איפה הוא נמצא), מזהה של הספרינט (איפה בדיוק הוא נמצא). (מזהה של פרויקט מטבלת ה-Projects).

SprintTasks – מאפשר אחסון מדויק יותר של ספרינטים עם משימות. ביניהם המזהה של המשימה ושל הספרינט. (מזהה של המשימה מטבלת ה-TASKS).

NAME	ID
Guest	1
Worker	2
Manager	3

Comments – מאפשר אחסון של תגובות על משימות. ביניהם המזהים של המשימה, של המשתמש ששלח את התגובה, תוכן התגובה ותאריך שליחתה. (מזהה של המשימה מטבלת ה-TASKS).

Files – מאפשר אחסון של קבצים בתגובות. ביניהם מזהה התגובה, ומיקום הקובץ. (מזהה של התגובה מטבלת ה-COMMENTS).

○ סקירת חולשות והאיומים שכבת האפליקציה:

• חולשה SQL Injection :

כדי להתמודד בבעיה זאת, השתמשתי ב- Prepared Statements / Parameterized Queries, כלומר במקום לבנות מחרוזת SQL עם מחרוזות קלט גולמיות, השתמשתי בפרמטרים מאובטחים. דוגמא:

```
query = '''
        SELECT CLIENTS.ID
        FROM CLIENTS
        WHERE CLIENTS.Username = ? AND CLIENTS.EMAIL = ?
    '''
cursor = self.conn.execute(query, (username, email))
```

• **MITM**: כדי למנוע גניבת מפתח בעברתו מהשרת לשרת מסד הנתונים, השתמשתי בשיטת החלפת המפתחות באמצעות העברת מפתח בשיטה א-סימטרית RSA. פירוט על דרך בה עובר המפתח נמצא בסעיף "תיאור פרוטוקול תקשורת"

• **Web Session**: כדי לאפשר כל משתמש לעבוד בדפדפן משלו, כלומר בצורה מבודדת - בלי שתהיה "דריסת מידע" בין משתמשים שונים - השתמשתי ב-Session. ה-Session מאוחסן בצד ה-Backend, ומזהה כל משתמש לפי מזהה ייחודי (session_id) בתוך ה-Session ניתן לשמור מידע זמני כמו:

- ❖ שם משתמש (username)
- ❖ סיסמה (password)
- ❖ אימייל (email)
- ❖ מזהה של משתמש (client_id)
- ❖ תפקיד (role)
- ❖ פרויקטים (projects)
- ❖ פרויקטים משותפים (shared_projects)
- ❖ ספרינטים (sprints)
- ❖ משימות (tasks)
- ❖ תמונת פרופיל (pic)
- ❖ חברים (friends)
- ❖ עובדים (workers)
- ❖ הודעות (notifications)

כך ניתן לשמור מידע זמני של המשתמש בין בקשות שונות, מבלי להעביר את המידע שוב ושוב או לפנות למסד הנתונים בכל פעולה.

שכבת התעבורה:

• כל מידע מוצפן ע"י הצפנה סימטרית AES-128, כלומר מפתח בעל 16 bytes.

מודולים מיובאים:

- ❖ **socket** - מיועד לתקשורת בין מחשבים ברשת באמצעות פרוטוקול TCP.
- ❖ **flask** - מיועדת לפיתוח WebApplication.
- ❖ **flask_session** - מיועד לניהול Session בצד ה-BackEnd (השרת), כדי לשמור מידע על ושל המשתמש בצורה בטוחה ומתמשכת.
- ❖ **flask_socketio** - מיועדת לניהול תקשורת בזמן אמת בין ה-FrontEnd (הדפדפן) לבין ה-BackEnd, באמצעות WebSocket.
- ❖ **threading** - מאפשר להריץ חלקים מהקוד במקביל, כדי לבצע פעולות רקע בלי לעכב את הפעולה הראשית של התוכנית.
- ❖ **datetime** - משמש לעבודה עם תאריכים, שעות וחישובים ביניהם – כולל תאריך נוכחי, חישוב זמן שעבר, פורמטים מותאמים ועוד.
- ❖ **Optional** **from typing import** - מאפשר להגדיר שפרמטר או ערך כלשהו יכול להיות מסוג מסוים או – None כלומר, הוא לא חובה שהוא יהיה מוגדר.
- ❖ **mimetypes** - משמש לזיהוי סוגי קבצים לפי סיומת הקובץ. (דוגמה: image.jpg --> jpg)
- ❖ **uuid** - יוצר מזהים ייחודיים - שימושי לתיוג עצמים, קבצים או משתמשים בצורה שאינה מתנגשת.
- ❖ **json** - מאפשר להמיר בין אובייקטים של פייתון (כמו מילונים) לבין מחרוזות JSON - נפוץ מאוד בתקשורת בין FrontEnd ל-BackEnd אצלי בפרויקט.
- ❖ **base64** - משמש לקידוד או פענוח של מידע בבסיס 64 – לדוגמה קבצים בינאריים, תמונות או טוקנים בצורה טקסטואלית.
- ❖ **re** - משמש לעבודה עם ביטויים רגולריים (Regular Expressions) לצורך חיפוש, התאמה וסינון טקסטים לפי תבניות.
- ❖ **sqlite3** - מאפשר עבודה עם מסד נתונים מקומי ביצירת טבלאות, הוספת נתונים, שליפה ועוד, בלי צורך בשרת חיצוני.
- ❖ **hashlib** - משמש להצפנה חד-כיוונית (כמו hash-256) לשמירת סיסמאות בצורה מאובטחת.
- ❖ **cryptography** - ספרייה מתקדמת להצפנה וקריפטוגרפיה – תומכת בהצפנה סימטרית, א-סימטרית.
- ❖ **from Crypto.Cipher import PKCS1_OAEP** - מאפשר הצפנה/פענוח בטוחים עם RSA לפי תקן PKCS#1 v2 (שיטה עם הגנה משופרת נגד התקפות).
- ❖ **from Crypto.PublicKey import RSA** - מאפשרת ליצור ולנהל מפתחות RSA להצפנה א-סימטרית (מפתח ציבורי ופרטי).
- ❖ **from Crypto import Random** - משמש ליצירת ערכים רנדומליים קריפטוגרפיים שנדרשים להצפנה בטוחה.

מודולים / מחלקות שפותחו בפרויקט:

- ❖ **Security.Encryption/Security.Asymmetric** - מיועד ליבוא מחלקה אישית בשם Encryption או בשם Asymmetric מתוך ספרייה בשם Security, המאפשרת שימוש בהצפנה ופיענוח. (בצד הלקוח) קבלת מפתח/יצירת ציבורי מפתח, ואף קביעת מפתח ההצפנה הפרטי (בצד השרת).
- ❖ **from Database.DatabaseParent import DatabaseParent** - מחלקה ראשית המכילה מספר מחלקות שקשורות למסד נתונים (קובץ אחד) ומאפשרת לטבלאות שונות להיווצר תחת מחלקות שונות אך עדיין להצביע על אותו הקובץ מסוג db.
- ❖ **from Database.DatabaseClients import DatabaseClients** - מחלקה היוצרת מספר טבלאות במסד הנתונים המכילות סוגי מידע שונים על המשתמשים, כמו משתמשים, הגדרת התפקיד, חברים, עובדים.
- ❖ **from Database.DatabaseProjects import DatabaseProjects** - מחלקה היוצרת מספר טבלאות במסד הנתונים המכילות סוגי מידע שונים על הפרויקטים. כמו פרויקטים ופרויקטים משותפים.
- ❖ **from Database.DatabaseTasks import DatabaseTasks** - מחלקה היוצרת מספר טבלאות במסד הנתונים המכילות סוגי מידע שונים על משימות. כמו משימות, ספרינטים, משותפים.

❖ `from ConnectionWithDatabase import ConnectionWithDatabase` – מחלקה המאפשרת חיבור עם

שרת מסד נתונים.

❖ `Network` – מחלקה ייצרת/הפעלת השרת ראשי יחד עם הגישה למסד הנתונים

שם המחלקה: `ConnectionWithDatabase`

תפקיד המחלקה: מהווה גשר בין שרת ה-Flask לבין שרת מסד נתונים ומקיים חיבור TCP עם שרת מסד הנתונים כולל הצפנה. מטפל בחיבור מבוסס socket בין לקוח לשרת מסד נתונים. הוא מקים תקשורת מוצפנת באמצעות RSA להחלפת מפתחות, ו-AES להצפנת הודעות, המאפשר שליחה וקבלה של הודעות.

תכונות המחלקה:

`user_id` - מזהה משתמש ייחודי לצורך מעקב אישי בחיבור עם השרת.
`socket` - האובייקט האחראי על החיבור TCP המשמש לתקשורת עם שרת מסד הנתונים.
`enc` – אובייקט ההצפנה Encryption
`connected` - בוליאני המציין אם החיבור לשרת פעיל.

פעולות במחלקה:

`connect(self)`

טענת כניסה: אין

טענת יציאה: יוצר חיבור לשרת אם עדיין לא מחובר, מבצע החלפת מפתחות ושולח הודעת ברכה ראשונית ("HELLO_")

`key_exchange(self)`

טענת כניסה: אין

טענת יציאה: מקבל את מפתח ה-RSA הציבורי של השרת, משתמש בו כדי להצפין את מפתח ה-RSA ושולח את מפתח ה-AES המוצפן בחזרה לשרת כדי לקיים תקשורת (מוצפנת) משותפת.

`send(self, message)`

טענת כניסה: `message` (מחרוזת) - הודעת טקסט רגיל לשלוח לשרת.

טענת יציאה: מצפין את ההודעה באמצעות AES, מקדים לה את אורכה ושולח אותה דרך ה-socket.

`receive(self)`

טענת כניסה: אין

טענת יציאה: מקבל הודעה מוצפנת מהשרת, מפענח אותה ומחזיר את ההודעה המפוענחת שהתקבלה. אם החיבור נסגר או נכשל, מחזיר "ENDED" וסוגר את התקשורת.

`close(self)`

טענת כניסה: אין

טענת יציאה: סוגרת את החיבור בצורה תקינה ושולח הודעת ניתוק "END" לשרת.

שם המחלקה: `Network`

תפקיד המחלקה: שרת TCP שמטפל בשרת ריבוי לקוחות, מקבל בקשות מלקוחות ומבצע פעולות על מסד הנתונים באמצעות תקשורת מוצפנת RSA ו-AES בין לקוחות ומסד נתונים.

תכונות המחלקה:

`database` - אובייקט מסד נתונים.

`enc` - אובייקט ההצפנה Encryption

`server_socket` - שקע שרת המאזין לחיבורים נכנסים מלקוחות TCP.

`clients` - מילון (dictionary) המאחסן את המידע של כל לקוח מחובר – כתובת, מפתח AES, אובייקט הצפנה

אישי ועוד.

sockets_list - רשימת כל השקעים (sockets) שמנוטרים ע"י select - כולל שקע השרת ושקעי הלקוחות.

פעולות במחלקה:

run_server(self)

טענת כניסה: אין

טענת יציאה: מאזין לחיבורים נכנסים באמצעות לולאה עם select לניהול מספר לקוחות בו זמנית. מקבל חיבורים,

מבצע החלפת מפתחות ומנתב הודעות ל- handle_message().

key_exchange(self, client_socket)

טענת כניסה: socket של הלקוח שהתחבר לשרת.

טענת יציאה: שולח מפתח RSA ציבורי, מקבל מפתח AES מוצפן מהלקוח, מפענח אותו ומחזיר אובייקט הצפנה

שאותחל עם מפתח AES.

handle_messages(self, client_socket, message)

טענת כניסה: client_socket - socket של הלקוח ששלח את ההודעה, message - ההודעה/הפקודה שנשלחה.

טענת יציאה: מנתח, מעבד פקודות ושולח תגובות מתאימות באמצעות send().

send(self, client_socket, message)

טענת כניסה: client_socket - socket של הלקוח שאליו תשלח ההודעה, message - ההודעה שתוצפן ולבסוף

תשלח.

טענת יציאה: מצפין את ההודעה באמצעות מפתח AES של הלקוח ושולח אותה עם קידומת באורך 4 בתיים.

receive(self, socket)

טענת כניסה: socket - socket של הלקוח שממנו תתקבל ההודעה.

טענת יציאה: קוראת 4 בתיים (אורך ההודעה), ואז קוראת את כל ההודעה המוצפנת, מפענחת ומחזירה מחרוזת

טקסט. מחזירה None אם יש ניתוק או שגיאה.

Disconnect_client(client_socket, socket_list)

טענת כניסה: client_socket - ה-socket שרוצים לנתק. רשימת ה-sockets הפעילים.

טענת יציאה: מסיר את socket הלקוח מהמעקב, סוגר את החיבור ומוחק את הנתונים הזמניים שמשייכים בתהליך

התקשורת.

שם המחלקה: DatabaseParent

תפקיד המחלקה: המחלקה משמשת כשכבת ניהול מרכזית לכל הפעולות מול מסד הנתונים.

היא יוצרת חיבור ל-SQLite ומרכזת גישה לטבלאות projects, clients ו-tasks באמצעות מחלקות משנה.

היא מאפשרת ביצוע פעולות קריאה וכתבייה שונות בצורה מרוכזת עבור צד השרת.

תכונות המחלקה:

tables - מילון המכיל מופעים של מחלקות ניהול טבלאות (DatabaseTasks, DatabaseProjects, DatabaseClients).

כל טבלה מקבלת את אותו חיבור למסד הנתונים.

פעולות במחלקה:

__init__(self)

טענת כניסה: אין

טענת יציאה: אתחול האובייקט - יוצר חיבור למסד הנתונים wowdatabase.db ומאתחל את שלוש הטבלאות

המרכזיות דרך מחלקות-משנה.

login(self, username, password)

טענת כניסה: מחרוזות: שם משתמש וסיסמה

טענת יציאה: מבצע התחברות של לקוח דרך טבלת client. מחזיר האם הצליח, מזהה הלקוח והתפקיד שלו (role).

`read_list_by_ID(self, client_id, project_id, name)`
 טענת כניסה: מזהה לקוח, מזהה פרויקט, וסוג נתון (comments או sprints)
 טענת יציאה: מחזיר רשימות לפי הקשר - מטלות בספרינט או תגובות למשימה מסוימת.

`addTask(self, client_id, project_id, sprint_id, task_id, name)`
 טענת כניסה: מקבל מזהים שונים מסוג (int) ושם משימה (string).
 טענת יציאה: מוסיף משימה ריקה לפרויקט ולספרינט המבוקש.

`delete(self, subject, ids)`
 טענת כניסה: subject – מחרוזת מזהה את הסוג האובייקט למחיקה (tasks, projects), ו-ids רשימת מזהים.
 טענת יציאה: מוחק פרויקט או משימה לפי סוג ומזהים.

שם המחלקה: DatabaseClients

תפקיד המחלקה: מחלקה זו מנהלת את בסיס הנתונים של המערכת, כולל משתמשים (לקוחות), חברים, בקשות חברות, תפקידים (Roles), עובדים, ובקשות עובדים. המחלקה אחראית על יצירת הטבלאות, רישום והתחברות של משתמשים, ניהול חברים ועובדים, וטיפול בבקשות.

תכונות המחלקה:

conn - אובייקט חיבור למסד הנתונים (SQLite). משמש לביצוע של שליפה של מידע וגם שינויים במסד הנתונים

פעולות במחלקה:

`__init__(self, conn)`

טענת כניסה: אין.

טענת יציאה: יוצר את מבנה בסיס הנתונים על ידי קריאה לפונקציית `createDb`.

`createDb(self)`

טענת כניסה: אין.

טענת יציאה: יוצר את כל הטבלאות הרלוונטיות בבסיס הנתונים אם הן לא קיימות.

`addClient(self, username, passw, email)`

טענת כניסה: שם משתמש, סיסמה, ודוא"ל. (מחרוזות).

טענת יציאה: מוסיף משתמש חדש אם הוא לא קיים. מחזיר True אם הצליח, False אחרת.

`loginClient(self, username, passw)`

טענת כניסה: שם משתמש וסיסמה (מחרוזות).

טענת יציאה: מחזיר True, מזהה משתמש ותפקיד אם ההתחברות הצליחה; אחרת False, None, None.

`addFriend(self, client_id, friend_id)`

טענת כניסה: מזהי שני משתמשים (int).

טענת יציאה: מוסיף קשר חברות הדדי. מחזיר True אם הצליח, False אחרת.

`removeFriend(self, sender_id, receiver_email)`

טענת כניסה: מזהה השולח (int) ודוא"ל של המקבל (מחרוזת).

טענת יציאה: שולח בקשת חברות אם ניתן. מחזיר True אם הצליח, False אחרת.

`addFriendRequest(self, client_id, friend_username)`

טענת כניסה: מזהה משתמש (int) ושם המשתמש של החבר המקבל (מחרוזת)

טענת יציאה: מסיר את הקשר בין המשתמש לחברו.

`acceptFriendRequest(self, client_id, friend_username)`
 טענת כניסה: מזהה משתמש (int) ושם המשתמש של החבר המקבל (מחרוזת)
 טענת יציאה: מקבל את הבקשה ויוצר קשר חברות הדדי. מחזיר את שם המשתמש של המקבל אם הצליח, None אחרת.

`listFriend(self, client_id, online_users)`
 טענת כניסה: מזהה משתמש (int) ומילון מזהי משתמשים מחוברים (dict).
 טענת יציאה: מחזיר רשימת חברים בפורמט JSON עם פרטים, כולל האם כל חבר מחובר.

`getUserByEmail(self, email)`
 טענת כניסה: כתובת דוא"ל (מחרוזת).
 טענת יציאה: מחזיר (ID, Username) אם נמצא, אחרת None.

`getUserById(self, id)`
 טענת כניסה: מזהה משתמש (int).
 טענת יציאה: מחזיר מחרוזת המכילה את כתובת הדוא"ל של המשתמש, או None אם נמצא.

`getPendingFriendRequests(self, receiver_email)`
 טענת כניסה: כתובת דוא"ל של המקבל (מחרוזת).
 טענת יציאה: מחזיר רשימת בקשות חברות ממתינות בפורמט JSON.

`addWorker(self, owner_id, worker_email)`
 טענת כניסה: מזהה בעל העסק (int) ודוא"ל של העובד (מחרוזת).
 טענת יציאה: מוסיף את המשתמש כעובד. מחזיר True אם הצליח, False אם העובד כבר קיים או לא נמצא.

`removeWorker(self, owner_id, worker_username)`
 טענת כניסה: מזהה בעל העסק (int) ושם משתמש של העובד (מחרוזת).
 טענת יציאה: מסיר את הקשר בין בעל העסק לעובד.

`addWorkerRequest(self, sender_id, receiver_email)`
 טענת כניסה: מזהה השולח (int) ודוא"ל המקבל (מחרוזת).
 טענת יציאה: שולח בקשת עובד אם המשתמש נמצא. מדפיס הודעת הצלחה או כישלון.

`getPendingWorkerRequests(self, receiver_email)`
 טענת כניסה: כתובת הדוא"ל של מקבל הבקשות (מחרוזת).
 טענת יציאה: מחזירה רשימה של בקשות עובד ממתינות למייל הנתון. כל פריט הוא מילון עם:
 sender_id : מזהה השולח
 sender_username : שם המשתמש של השולח
 type: "work request"

`acceptWorkerRequest(self, sender_id, receiver_email)`
 טענת כניסה: מזהה השולח של הבקשה (int), כתובת הדוא"ל של מקבל הבקשה (מחרוזת).
 טענת יציאה: אם המקבל נמצא במסד הנתונים, הפעולה יוצרת קשר עובד בין השולח למקבל. מעדכנת את סטטוס הבקשה ל- "Accepted". מחזירה את שם המשתמש של המקבל (receiver_username). אחרת — מחזירה None.

```
get_workers(self, manager_id)
```

טענת כניסה: מזהה המנהל שעובדיו נשלפים (int).

טענת יציאה: מחזירה רשימת מילונים, כאשר כל מילון מכיל מידע על עובד:

id : מזהה המשתמש

username : שם משתמש

email : דוא"ל

role : תפקיד (מסויך ל- 2 = ROLE_ID)

project_count : מספר פרויקטים שהוא שותף או בעלים עליהם.

```
getPendingRequests(self, email)
```

טענת כניסה: כתובת הדוא"ל של המשתמש (מחוזת).

טענת יציאה: מחזירה רשימה עם מילון יחיד שמכיל:

```
}
```

רשימת בקשות חברות ממתיונות # [...], "friends":

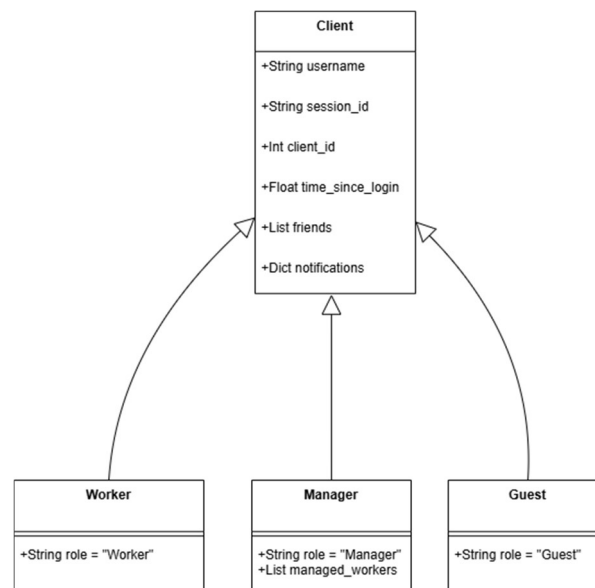
רשימת בקשות עובד ממתיונות # [...], "workers":

```
{
```

בנוסף, קיימת גם ירושה. בה נוצרת אובייקט מסוג לקוח (Client) עם הפרמטרים של session_id, שם המשתמש (username), ומזהה הלקוח.

סוג הירושה שיתקבל תלוי בתפקידו של המשתמש.

להלן הירושה;



חלק ב' –

ניהול משימות בתוך פרויקט : עדכון משימה.

בעדכון המשימות. אני מחויב לאפשר למשתמש לעדכן את המשימה, לקשר את המשימה למסד הנתונים ואחריה לעדכן את הממשק הוויזואלי (דפדפן).

השלב של קליטת הנתונים מהדפדפן הייתה הכי מאתגרת בעיניי, משום שהיה צריך כל כך הרבה EventListeners ב-JavaScript.

להלן הקוד ב-Python:

```
@app.route("/update_task", methods=["POST"])
def update_task():
    try:
```



```

session_id = session["session_id"]
connection = clients[session_id]["connection"]
if not connection.connected:
    return render_template("404.html",
                           error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404

update_data = request.json
task_id = update_data.get("ID")
sprint_id = update_data.get("SprintID")
task_name = update_data.get("TaskName", "").strip()

project_id = update_data.get("project_id") or
session.get("project_id")
username = session.get("username")
if not task_id or not sprint_id or not project_id:
    return jsonify({"status": "error", "message": "Task ID, Sprint
ID, and Project ID are required"}), 400

print("Update data received:", update_data)
print("Session sprints:", session.get("sprints", []))

sprint_name = ""
task = None
# region Locate the task
for sprint in session.get("sprints", []):
    if int(sprint.get("SprintID")) == int(sprint_id):
        sprint_name = sprint["SprintName"]
        for t in sprint.get("Tasks", []):
            if int(t.get("TaskID")) == int(task_id):
                task = t
                break

if not task:
    return jsonify({"status": "error", "message": "Task not
found"}), 404
# endregion

# region Updating
# Track changes for logging
change_log = []

# Validate and update task name
if task_name:
    if task["TaskName"] != task_name:
        change_log.append(f"Task name changed from
'{task['TaskName']}' to '{task_name}'")
        task["TaskName"] = task_name

task = updatePlus(task, update_data)
# endregion

socketio.emit("task_updated", {
    "project_id": project_id,
    "sprint_id": sprint_id,
    "updated_task": task,
    "changes": change_log,
    "updated_by": username,

```

```

        "sprint_name": sprint_name
    }, room=f"project_{project_id}")

    print(f"\n\nUpdated Task: {task}\n\n")

    # Update the session with the modified task
    for sprint in session.get("sprints", []):
        if int(sprint["SprintID"]) == int(sprint_id):
            for i, task_ in enumerate(sprint["Tasks"]):
                if int(task_["TaskID"]) == int(task["TaskID"]):
                    sprint["Tasks"][i] = task # Directly update task
                    session.modified = True
                    break

    print("Change log:", change_log)

    # Print updated sprint tasks
    for sprint in session.get("sprints", []):
        print(f"Tasks in Sprint {sprint['SprintID']}:
{sprint.get('Tasks', [])}")

    # TODO: if the date is closed to the DueDate, the system would add a
notification to the user about it.

    # Notify the client through WebSocket
    if session.get("session_id") in clients:
        try:
            clients[session["session_id"]]["connection"].send(f"UPDATE_TASK${task}")
            response =
clients[session["session_id"]]["connection"].receive() # No timeout
parameter
            print("WebSocket response:", response)
        except Exception as e:
            print(f"WebSocket error: {e}")

        return jsonify({"status": "success", "updated_task": task,
"changes": change_log})
    except Exception:
        return render_template("404.html", error="You made a mistake,
buddy"), 404

def updatePlus(task, update_data):
    today = datetime.today()
    new_due_date_str = update_data.get("DueDate")
    current_due_date_obj = parse_due_date(task.get("DueDate"))
    new_due_date_obj = parse_due_date(new_due_date_str) if new_due_date_str
else None

    # Only update if DueDate is different
    if new_due_date_obj and current_due_date_obj != new_due_date_obj:
        task["DueDate"] = new_due_date_obj.strftime("%d/%m/%Y") # Store in
same format

    due_date = parse_due_date(task.get("DueDate")) # re-parse just to be
safe
    days_left = (due_date - today).days if due_date else None
    print("Days left =", days_left)

```

```
# --- Priority --- #
if not update_data.get("Priority"):
    task["Priority"] = infer_priority(due_date, today)
    task["PriorityColor"] = PRIORITY_COLORS.get(task["Priority"],
"#9ca3af")
elif task["Priority"] != update_data["Priority"]:
    task["Priority"] = update_data["Priority"]
    if "PriorityColor" in update_data:
        task["PriorityColor"] = update_data["PriorityColor"]

# --- Status --- #
if not update_data.get("Status"):
    task["Status"] = infer_status(due_date, today)
    task["StatusColor"] = STATUS_COLORS.get(task["Status"], "#d1d5db")
elif task["Status"] != update_data["Status"]:
    task["Status"] = update_data["Status"]
    if "StatusColor" in update_data:
        task["StatusColor"] = update_data["StatusColor"]

# --- Progress --- #
if "Progress" in update_data:
    try:
        task["Progress"] = int(update_data["Progress"])
    except (ValueError, TypeError):
        task["Progress"] = 0
else:
    task["Progress"] = infer_progress(task["Status"], due_date, today)
    align_priority_with_status(task, days_left)
return task
```

להלן החלק של JavaScript

```
document.addEventListener("DOMContentLoaded", function () {
    let debounceTimer;

    // ----- REAL-TIME TASK NAME UPDATE -----
    \
    document.querySelectorAll(".task-name-cell").forEach(cell => {
        const input = cell.querySelector(".edit-input");
        const displayText = cell.querySelector(".display-text");

        displayText.addEventListener("click", function () {
            displayText.style.display = "none";
            input.style.display = "block";
            input.value = displayText.textContent.trim();
            input.focus();
        });

        input.addEventListener("focus", function () {
            input.dataset.active = "true";
        });

        input.addEventListener("blur", function () {
            input.dataset.active = "false";
            setTimeout(() => {
                if (input.dataset.active === "false") {
                    input.style.display = "none";
                }
            }, 100);
        });
    });
});
```

```

        displayText.textContent = input.value.trim() || "Untitled
Task";

        displayText.style.display = "inline";
    }
    }, 100);
});

input.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        const taskRow = this.closest(".task-row");
        const taskId = taskRow.id.replace("task-", "");
        const sprintId = taskRow.closest("tbody").id.replace("taskBody-",
        "", "");

        const newTaskName = this.value.trim();
        if (newTaskName) {
            updateTask(sprintId, taskId, { TaskName: newTaskName });
        }
        this.style.display = "none";
        displayText.textContent = newTaskName || "Untitled Task";
        displayText.style.display = "inline";
    }
});

// debounce שם המשימה עם
input.addEventListener("input", function () {
    clearTimeout(debounceTimer);
    const taskRow = this.closest(".task-row");
    const taskId = taskRow.id.replace("task-", "");
    const sprintId = taskRow.closest("tbody").id.replace("taskBody-",
    "");

    const newTaskName = this.value.trim();
    if (newTaskName) {
        debounceTimer = setTimeout(() => {
            updateTask(sprintId, taskId, { TaskName: newTaskName });
        }, 500);
    }
});

// לחיצה מחוץ לשדה סוגרת את העריכה
document.addEventListener("click", function (event) {
    document.querySelectorAll(".edit-input").forEach(input => {
        if (!input.contains(event.target) && !input.closest(".task-name-
cell").contains(event.target)) {
            input.style.display = "none";
            const displayText = input.closest(".task-name-
cell").querySelector(".display-text");
            displayText.textContent = input.value.trim() || "Untitled Task";
            displayText.style.display = "inline";
        }
    });
});

// ----- DROP MENU FUNCTIONALITY ----- \\
document.querySelectorAll('.dropdown-Status, .dropdown-
Priority').forEach(element => {
    element.addEventListener('click', function (event) {
        event.stopPropagation();
        const field = this.dataset.field;
        const taskId = this.dataset.id;
        const menu = document.getElementById(`dropdown-${field}`);
    });
});

// סגירת תפריטים פתוחים אחרים

```

```

document.querySelectorAll('.dropdown-menu').forEach(otherMenu => {
    if (otherMenu !== menu) {
        otherMenu.classList.remove('show');
        otherMenu.classList.add('hide');
    }
});
// הסתרת/הצגת התפריט הנבחר
menu.classList.toggle('show');
menu.classList.toggle('hide');

const rect = this.getBoundingClientRect();
menu.style.left = `${rect.left}px`;
menu.style.top = `${rect.bottom + window.scrollY}px`;
// המשימה מזהה שמירת
menu.setAttribute('data-task-id', taskId);
});
});
// סגירת כל התפריטים בלחיצה מחוץ להם
document.addEventListener('click', function (event) {
    document.querySelectorAll('.dropdown-menu').forEach(menu => {
        if (!menu.contains(event.target)) {
            menu.classList.remove('show');
            menu.classList.add('hide');
        }
    });
});

// ----- HANDLE DROPDOWN OPTION SELECTION -----
// \
document.querySelectorAll('.dropdown-menu div').forEach(option => {
    option.addEventListener('click', function () {
        const menu = this.parentElement;
        const field = menu.id.replace('dropdown-', '');
        const selectedValue = this.getAttribute('data-value');
        const taskId = menu.getAttribute('data-task-id');
        const element = document.querySelector(`[data-
field="${field}"][data-id="${taskId}"]`);
        const sprintId = element.closest("tbody").id.replace("taskBody-",
        "");

        if (selectedValue === "Custom") {
            currentCustomField = field;
            currentTaskElement = element;
            currentTaskId = taskId;
            currentSprintId = sprintId;
            openCustomLabelModal(field);
        } else {
            updateField(sprintId, taskId, field, selectedValue, element);
        }
    });
});

// ----- GLOBAL VARIABLES ----- \
var currentCustomField = null;
var currentTaskElement = null;
var currentSprintId = null;
var currentTaskId = null;

// ----- FUNCTION TO UPDATE STATUS OR PRIORITY -----
// \

```

```
function updateField(sprintId, taskId, field, value, element) {
  const STATUS_COLORS = {
    "Ready to Start": "#d0ebff",
    "In Progress": "#facc15",
    "Waiting for Review": "#cfe2ff",
    "Updating": "#dbeafe",
    "Pending Deploy": "#e8e4d3",
    "Done": "#d1fae5",
    "Future Plan": "#baf5e0",
    "Working On It": "#e9d5ff",
    "Stuck": "#fca5a5",
    "In Tests": "#d9f99d",
    "": "#d1d5db"
  };

  const PRIORITY_COLORS = {
    "Critical": "#f87171",
    "High": "#facc15",
    "Medium": "#bfdbfe",
    "Low": "#d1fae5",
    "Best Effort": "#e5e7eb",
    "Missing": "#9ca3af"
  };

  const customColors = JSON.parse(localStorage.getItem("customColors")) || {};

  let colors = field === "Priority" ? PRIORITY_COLORS : STATUS_COLORS;
  let newColor = colors[value] || customColors[value];

  if (!newColor) {
    newColor = generateRandomPastelColor();
    customColors[value] = newColor;
    localStorage.setItem("customColors", JSON.stringify(customColors));
  }

  element.style.backgroundColor = newColor;
  element.style.color = "#333";
  element.innerText = value;

  updateTask(sprintId, taskId, { [field]: value, [`_${field}Color`]: newColor });

  document.querySelectorAll('.dropdown-menu').forEach(menu => {
    menu.classList.remove('show');
    menu.classList.add('hide');
  });
}

// ----- RANDOM PASTEL COLOR ----- \\
function generateRandomPastelColor() {
  const hue = Math.floor(Math.random() * 360);
  return `hsl(${hue}, 80%, 85%)`;
}

// ----- BACKEND UPDATE FUNCTION ----- \\
export async function updateTask(sprintId, taskId, updateData) {
  updateData["ID"] = taskId;
  updateData["SprintID"] = sprintId;
  updateData["project_id"] = document.body.getAttribute('data-project-id');

  console.log("Calling updateTask with:", {
```

```

        taskId,
        sprintId,
        projectId: document.body.getAttribute('data-project-id'),
        updateData
    });

    try {
        const response = await fetch("/update_task", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(updateData)
        });
        const result = await response.json();

        if (result.status === "success") {
            console.log("Task updated:", result.updated_task);
            const updatedTask = result.updated_task;
            const field = Object.keys(updateData).find(k => k !== "ID" && k !==
"SprintID" && k !== "project_id");
            const element = document.querySelector(`[data-
field="${field}"][data-id="${taskId}"]`);
            if (element) {
                const colorKey = `${field}Color`;
                const newColor = updatedTask[colorKey] || "#d1d5db";
                element.style.backgroundColor = newColor;
                element.style.color = "#333";
            }
        } else {
            console.error("Failed to update task:", result.message);
        }
    } catch (error) {
        console.error("Error updating task:", error);
    }
}

// ----- CUSTOM LABEL MODAL ----- \\
function openCustomLabelModal(field) {
    currentCustomField = field;
    document.getElementById("customField").innerText = field;
    document.getElementById("customLabelName").value = "";
    document.getElementById("customLabelColor").value = "#d1d5db";
    document.getElementById("customLabelModal").classList.remove("hide");
    document.getElementById("customLabelModal").classList.add("show");
}

function closeCustomLabelModal() {
    document.getElementById("customLabelModal").classList.remove("show");
    document.getElementById("customLabelModal").classList.add("hide");
}

function saveCustomLabel() {
    const labelName = document.getElementById("customLabelName").value.trim();
    const labelColor = document.getElementById("customLabelColor").value;
    if (!labelName) {
        alert("Please enter a label name.");
        return;
    }
    if (currentTaskElement) {
        currentTaskElement.style.backgroundColor = labelColor;
        currentTaskElement.innerText = labelName;
    }
}

```

```
updateTask(currentSprintId, currentTaskId, { [currentCustomField]:
labelName, [`${currentCustomField}Color`]: labelColor });
}
closeCustomLabelModal();
}
```

חלק ג' –

מסמך בדיקות

מטרת הבדיקה	מה בוצע בפועל	תוצאות הבדיקה	במידה והתגלו בעיות
לבדוק שהוספת משימה שומרת נתונים כראוי	הופעלה פונקציה add_task() עם פרטי משימה תקינים ונבדק שנשמרו במסד הנתונים	המשימה נוספה בהצלחה, והמידע תועד במסד	הוחזרה למשתמש הודעת שגיאה (alert)
לבדוק טיפול בשדות חסרים בעת הוספת משימה	הופעלה הפונקציה ללא שדה חובה (למשל ללא שם משימה)	הוחזרה שגיאה צפויה ("שם משימה נדרש")	-
לודא שקיים חיבור בין ה-frontend לבין ה-backend	נלקחו נתונים מה-Frontend ונבדק האם הם מגיעים ל-Backend ולמסד הנתונים בהצלחה.	אם הוחזרה תשובה חיובית – המערכת ממשיכה לפעול תקין. אם הוחזרה תשובה שלילית – תוזרר למשתמש הודעת שגיאה מתאימה.	אם לא הוחזרה תשובה – ייתכן והתרחשה התנתקות מהשרת; המשתמש יקבל הודעת שגיאה ויתבקש להתחבר מחדש
ווידוא שהממשק ברור, מגיב נכון וידידותי	המשתמש ניתב בין המסכים, ביצע פעולות בסיסיות (התחברות, ניווט תפריטים)	אם הפעולות הצליחו והממשק הגיב כמצופה – המערכת ממשיכה לפעול באופן תקין.	אם התגלתה שגיאה – המשתמש מועבר לעמוד שגיאה (404), ובחלק מהמקרים נדרש להתחבר מחדש.

מדריך למשתמש

עץ קבצים

WOWDatabase side:

ROOT	/Database	/Security
- Main.py - Network.py - wowdatabase.db - Installer.py	- DatabaseClients.py - DatabaseParent.py - DatabaseProjects.py - DatabaseTasks.py __init__.py	- Asymmetric.py - Encryption.py

WOWProject:

ROOT	/Static	/CSS	/JS	avatars
- Client.py - ConnectionWithDatabase.py - Server.py	- avatar.jpg +	- AFK_Time.css - disable_click.css - listToDo.css	- AFK_Time.js - disable_double_right_click.js	- avatar_1.png - avatar_2.png - avatar_3.png

<pre> Installer.py +---Security Encryption.py +---static \---templates 404.html AFK_Time.html assign_member.html comments.html home.html HomeTasks_.html listToDo.html login.html my_workers.html popup.html profile.html projects.html settings.html sidebar.html task_context_menu.html team_dashboard.html topMenu.html </pre>	<pre> +---JS +---profile_pics 1.png 2.png 3.png 4.png o.png +---avatars </pre>	<pre> login.css style.css task_context_menu.css +---HomeTasks Main.css Table & Sprints.css workers.css \---Tasks Add_Task.css New_Task.css Task_DueDate.css Task_Name.css </pre>	<pre> disable_left_click.js disable_right_click.js friends.js HomeTasks.js javascript.js listToDo.js notifications.js +---Sprints Sprint_Context_Menu.js \---Tasks Add_Task.js Delete_Task.js DueDate_Task.js Socket_Room.js Socket_Task.js Update_Task.js Workers.js </pre>	<pre> avatar_4.png avatar_5.png avatar_6.png avatar_7.png avatar_8.png avatar_9.png \---Noam avatar.png </pre>
---	--	--	--	--

○ התקנת המערכת

כדי להריץ את התוכנית צריך להשתמש בסביבה שיוצעת להריץ פייתון, ההמלצה שלי היא להשתמש ב-Pycharm משום שזאת סביבת העבודה, שלדעתי נוחה.
 כדי להריץ את המערכת, צריך קודם להריץ את מוריד הסיפירות – Installer.py
 המיקומים בהם כל קובץ אמור להיות, הוא כמו שמוצג בעץ הקבצים לעיל.
 כדי שהפרויקט יעבוד, כל המחשבים צריכים להיות עם firewall off על מנת לאפשר חיבור של דפדפן האינטרנט עם המחשבים. **למידע מפורט יותר יש להסתכל בסעיף "משתמשי המערכת"**

הפרויקט הב גיט - <https://github.com/Kingman199/projct.git>

○ משתמשי המערכת

רשום בצורה מפורטת בעברית ובאנגלית בקישורים:
 אנגלית: [README_EN.md](#)
 עברית: [README_HE.md](#)

○ צילומי מסכי הפרויקט הרלוונטיים

בסעיף **לתיאור המסכים** ניתנו גם הוראות למשתמש המלווים בשפה פשוטה.

רפלקציה

● תיאור תהליך העבודה על הפרויקט

בתחילת השנה הגדרתי מטרה ברורה לפרויקט, אך במהלך העבודה הבנתי שהתהליך מורכב הרבה יותר ממה שצפיתי. היו שלבים של התקדמות מהירה, בעיקר כשתהליכי העבודה עבדו בצורה תקינה, לצד שלבים של עצירה והתמודדות עם בעיות לא צפויות. האתגרים כללו בין השאר: תקלות טכניות, חוסר ידע בתחומים מסוימים וניהול זמן מורכב ועמוס במשימות נוספות. עם הזמן, למדתי לחלק את העבודה למשימות קטנות, להציב מטרות ברורות לכל שבוע ולבצע בדיקות תקופתיות להתקדמות הפרויקט כדי לוודא שהכל עובד, מוכן ונראה "פיקס" גם מבחינה וויזואלית וגם מבחינת התוכן.

● הצלחות, אתגרים ודרכי פתרון

ההצלחה הבולטת ביותר הייתה השגת המטרה המרכזית בפרויקט – **עבודה בו-זמנית בין משתמשים** – לאחר עבודה ממושכת של ניסוי וטעייה. היו לי ימים שהייתי בטוח שהרעיון הזה גדול עליי, שאני סתם מגזים ודורש מעצמי יותר מידי. אחד האתגרים המרכזיים היה ללמוד טכנולוגיות חדשות תוך כדי העבודה, ללמוד שפות תכנות כמו - JavaScript, HTML, או שפה פשוטה כמו CSS. כדי להתמודד עם זה, השתמשתי במדריכים באינטרנט, פורומים, סרטונים ולעיתים פניתי למורים וחברים לקבלת ייעוץ ממוקד יותר.

- **תהליך הלמידה**
במהלך העבודה למדתי תכנים שלא הכרתי קודם גם ברמה הטכנית וגם בתכנון וניהול פרויקט. התנסיתי בעבודה עם כלים חדשים, תכנות בשפות שלא עבדתי איתן בעבר ושיטות עבודה, כמו: ניהול גרסאות, עבודה עם Flask, או יצירת ממשק משתמש נגיש, פשוט וידידותי לשימוש. הלמידה נעשתה בעיקר באופן עצמאי, אך גם תוך כדי התלבטות שלי עם חבריי והמורים המקצועיים שהנחו אותי.
- **כלים להמשך הדרך**
מהתהליך אני לוקח איתי כלים חשובים להמשך: ליצור ולתכנן פרויקט באמצעות "מחשבה יוצרת מציאות", יכולת חקר עצמאית, יכולת התמדה מול קשיים ואתגרים מורכבים, יכולת לעבוד לבד וללמוד בעצמי תכנים חדשים ויכולת לפרק בעיה מורכבת לחלקים פשוטים. בנוסף, למדתי את החשיבות של תיעוד בספר הפרויקט וניהול זמן נכון.
- **ראייה לאחר**
בדיעבד, ייתכן והייתי מקצה יותר זמן לשלב התכנון, כדי למנוע בזבז זמן בשלב הביצוע. בנוסף, הייתי מנסה לתעד את התהליך בצורה מסודרת יותר בספר – זה היה יכול לעזור בשלבים מתקדמים ולסייע בהצגת הפרויקט לאחר סיומו.
- **אם היו ברשותי משאבים נוספים**
עם יותר זמן או תקציב, הייתי מרחיב את הפרויקט – מוסיף לו שכבת שימושיות, כמו: אבטחה גבוהה יותר, משפר את חוויית המשתמש, או מוסיף יותר פיצ'רים שוויתרתי עליהם. גם שימוש בצידוד טכני מתקדם יותר או תוכנות בתשלום היו יכולות לשפר את התוצאות. יחד עם זאת אני גאה בתוצר הסופי שהגשתי.

תודות

אני רוצה להודות למורים על הסבלנות וההדרכה שליוות אותי לאורך כל הדרך: עידן פלדברג, עודי רז, קובי שוצמן וליאת סגל. למשפחה שתמכה בי גם בימים מלאי אכזבה. לחברים שסייעו, הקשיבו, ייעצו, החמיאו ונתנו ביקורת בונה כשצריך. תודה גם לכל מקור מידע ששימש אותי. תודה לעצמי שלא התייאשתי ועמדתי בציפיות שהצבתי לעצמי עם השקעה רבה, נחישות, התמדה והנאה מחוויית העבודה והיצירה.

ביבליוגרפיה

Flask-SocketIO. (n.d.). Flask-SocketIO documentation. Read the Docs.
<https://flask-socketio.readthedocs.io/en/latest/>

GeeksforGeeks. (n.d.). How to use Flask session in Python Flask?
<https://www.geeksforgeeks.org/how-to-use-flask-session-in-python-flask/>

GeeksforGeeks. (n.d.). Python SQLite.
<https://www.geeksforgeeks.org/python-sqlite/>

Pallets Projects. (n.d.). Flask API. Flask Documentation.
<https://flask.palletsprojects.com/en/stable/api/>

PythonBasics. (n.d.). Flask Sessions.
<https://pythonbasics.org/flask-sessions/>

Python Cryptographic Authority. (n.d.). Hazmat primitives – RSA. Cryptography Documentation.
<https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/>

Python Software Foundation. (n.d.). sqlite3 — DB-API 2.0 interface for SQLite databases. Python documentation.
<https://docs.python.org/3/library/sqlite3.html>

Python Software Foundation. (n.d.). typing — Support for type hints. Python documentation.
<https://docs.python.org/3/library/typing.html>

Tsabar, B. (2011, October). *MathBaseRSA*. Digital Whisper, (25).
<https://www.digitalwhisper.co.il/files/Zines/0x19/DW25-4-MathBaseRSA.pdf>

נספחים

server.py הוא השרת של ה-Flask (זה שמנהל את כל הבקשות מהמשתמש)
ConnectionWithDatabase.py – מגשר בין server.py לבין Network.py (מתנהג כמו לקוח)
Network.py הוא השרת המקורי של התקשורת.

SERVER.py

```
import threading, mimetypes, uuid, json, base64, re
from Client import *
from flask import Flask, jsonify, request, render_template, session,
```

```

redirect, url_for, send_file
from ConnectionWithDatabase import ConnectionWithDatabase
from flask_session import Session
from installer import main
from datetime import datetime
from typing import Optional

# region Handle Friend Requests in Real-Time
from flask_socketio import SocketIO, emit, join_room, leave_room
from functools import wraps

app = Flask(__name__, static_folder="static")
mimetypes.add_type('text/javascript', '.js')

# Force Flask to recognize JS as module-compatible

clients = {} # Store multiple user connections
clients_list = [] # Store multiple user connections
# session_id: ConnectionWithDatabase, [Client()]
client = None

# region Session
app.config["SECRET_KEY"] = "supersecretkey" # Must be set when
SESSION_USE_SIGNER is True
app.config["SESSION_TYPE"] = "filesystem"
app.config["SESSION_FILE_DIR"] = "./flask_session_data"
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_USE_SIGNER"] = True
Session(app)
# endregion

@app.route('/static/js/Tasks/DueDate_Task.js')
def serve_due_date_task():
    return send_file('static/js/Tasks/DueDate_Task.js',
mimetype='text/javascript')

def role_required(*allowed_roles):
    def wrapper(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            session_id = session.get("session_id")
            client = clients.get(session_id) ["client"]
            if not client or getattr(client, 'role', None) not in
allowed_roles:
                return redirect(url_for('404')) # Or some error page
            return f(*args, **kwargs)
        return decorated_function
    return wrapper

@app.before_request
def session_check():
    session.modified = True # Keeps the session active for permanent

```

```

sessions

    allowed_routes = ['login', 'static']
    if request.endpoint in allowed_routes:
        return

    if "time_since_login" in session:
        current_time = time.time()
        if current_time - session["time_since_login"] > 300: # 5
minutes
            session.clear()
            return redirect(url_for('login'))
        else:
            session["time_since_login"] = current_time # reset on
activity

# endregion

# Initialize Flask-SocketIO
socketio = SocketIO(app, cors_allowed_origins="*",
async_mode="threading") # Keep Flask as primary

# endregion

def run_socketio():
    socketio.run(app, host='0.0.0.0', port=5001, debug=False,
use_reloader=False, allow_unsafe_werkzeug=True)

@app.route('/')
def home():
    return render_template('home.html')

import os
@app.route('/login', methods=['GET', 'POST'])
def login():
    global clients, client

    if request.method == 'POST':
        try:
            username = request.form['username']
            password = request.form['password']

            temp_session_id = str(uuid.uuid4())
            connection = ConnectionWithDatabase(temp_session_id)

            if not connection.connected:
                return render_template('login.html', error="Database
connection failed.")

            connection.send(f"SIGNIN {username}#{password}")
            response = connection.receive()

```

```

        if "ENDED" in response:
            return render_template("404.html", error="Hold on,
buddy. Something is off...\nPlease 'wait' for the updates"), 404
        print("Login response:", response)

        if response and "User exists" in response:
            # Disconnect existing session
            if username in clients_list:
                print(f"User {username} already logged in.
Disconnecting old session.")
                old_session_id = next((sid for sid, data in
clients.items()
                                     if data.get("client") and
data["client"].username == username), None)
                if old_session_id:
                    clients[old_session_id]["connection"].close()
                    clients.pop(old_session_id, None)
                    clients_list.remove(username)

            # Parse ID and role
            parts = response.split(';')
            client_id = parts[0].split('ID:')[1].strip()
            role = parts[1].split('ROLE:')[1].strip()

            connection.send(f"GET_USER_EMAIL#{client_id}")
            email = connection.receive().strip()
            avatar_path =
f'static/profile_pics/{username}/avatar.png'

            # Determine which picture path to use
            if os.path.exists(avatar_path):
                # Use the user's custom avatar
                pic_path =
f"/static/profile_pics/{username}/avatar.png"
            else:
                # Fallback to a default or client-specific image
                pic_path =
f"/static/profile_pics/{client_id}.png"

            # Set session
            session.update({
                "username": username,
                "password": password,
                "session_id": temp_session_id,
                "client_id": client_id,
                "role": role,
                "pic": pic_path,
                "email": email
            })

            # Insert temporary client entry
            clients[temp_session_id] = {
                "connection": connection,

```

```

        "client": None
    }

    # Initialize client with role
    new_client = Role(role)
    clients[temp_session_id]["client"] = new_client
    session["time_since_login"] =
new_client.time_since_login
    clients_list.append(username)
    return redirect(url_for('projects_view',
session_id=temp_session_id))
    else:
        connection.close()
        return render_template('login.html', error="Invalid
username or password.")

    except Exception as e:
        import traceback
        print("🔥 Exception in /login:", e)
        traceback.print_exc()
        return render_template('login.html', error="Internal
Server Error. Please try again.")

    return render_template('login.html')

def Role(role):
    # Create the appropriate Client object
    if role == "Manager":
        session["workers"] =
fetch_workers_for_manager(session["client_id"])
        client_ = Manager(session["username"], session["session_id"],
session["client_id"])
        client_.managed_workers = session["workers"]
        return client_
    elif role == "Worker":
        return Worker(session["username"], session["session_id"],
session["client_id"])
    elif role == "Guest":
        return Guest(session["username"], session["session_id"],
session["client_id"])
    else:
        return Client(session["username"], session["session_id"],
session["client_id"])

@app.route('/projects')
def projects_view():
    try:
        if "session_id" not in session or "client_id" not in session:
            return redirect(url_for("login")) # Redirect to login if
session is missing

```

```

# Pull session_id from URL first
session_id = request.args.get("session_id") or
session.get("session_id")

client_id = clients[session_id]["client"].client_id
connection = clients[session_id]["connection"]
connection.send(f"GIVE PROJECTS#{client_id}")
proj = connection.receive()
except Exception:
    return render_template("404.html",
                           error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
    if "ENDED" in proj:
        return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404
    print("Raw received data:", proj)
    try:
        # if "Unknown command" in raw_proj:
        #     return "Error: Server did not recognize the request.",
500

    projects, share_project = proj.split('#', 1)[1].split('/', 1)
    # Ensure JSON format
    projects = json.loads(projects) # Parse JSON properly
    share_projects = json.loads(share_project) # Parse JSON
properly
    except (json.JSONDecodeError, IndexError) as e:
        print(f"JSON decoding error: {e}")
        return f"Error decoding project data: {e}", 500
    print(f"Shared Projects: {share_projects}")
    session["projects"] = projects
    session["shared_projects"] = share_projects
    session["notifications"] = get_notifications()

    return render_template(
        "projects.html",
        projects=session["projects"],
        shared_projects=session["shared_projects"],
        notifications=session["notifications"],
        session_id=session_id
    )

@app.route('/edit_project/<int:project_id>', methods=['POST'])
def edit_project(project_id):
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    updated_data = request.json
    print(f"Log: {updated_data}")
    client_id = clients[session_id]['client'].client_id
    connection = clients[session_id]["connection"]
    connection.send(f"UPDATE

```



```

projects#{client_id}.{project_id}#{updated_data}")
message = clients[session_id]["connection"].receive()
print(message)
project = None
for i in range(len(session["projects"])):
    if session["projects"][i]["id"] == project_id:
        session["projects"][i]["name"] = updated_data["name"]
        session["projects"][i]["description"] =
updated_data["description"]
        project = session["projects"][i]

    print(session["projects"])
    return jsonify({'message': 'Project updated', 'project':
project}), 200

@app.route('/add_project', methods=['POST'])
def add_project():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    data = request.get_json()
    project_type = data.get("type", "new")
    # session_id = session["session_id"]
    # client_id = session["client_id"]
    client_id = clients[session_id]["client"].client_id

    connection = clients[session_id]["connection"]
    # Tell the server to add a project
    if project_type == "new":
        connection.send(f"ADD_ITEM projects#{client_id}")
        message = connection.receive()
        if "ENDED" in message:
            return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404
        print(message)

        new_project = {
            'id': data['id'],
            'name': data['name'],
            'description': data['description']
        }
        session["projects"].append(new_project)
        return jsonify(new_project)

    elif project_type == "duplicate":
        print("Duplicating project...")
        original_project_id = data.get("original_project_id")
        print(f"Original project ID: {original_project_id}")
        original_project = next((p for p in session["projects"] if
p["id"] == original_project_id), None)

```

```

        if not original_project:
            print("Error: Original project not found")
            return jsonify({"error": "Original project not found"}),
404

        connection.send(f"DUPLICATE_ITEM
projects#{client_id}.{original_project_id}")
        response = connection.receive()
        if "ENDED" in response:
            return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404
        print(f"Server response: {response}")
        duplicated_project = {

            'id': data['id'],
            'name': original_project['name'] + " (Copy)",
            'description': original_project['description']
        }
        print(f"Duplicated project: {duplicated_project}")
        session["projects"].append(duplicated_project)

        return jsonify(duplicated_project)

@app.route('/delete_project', methods=['POST'])
def delete_project():
    data = request.json
    project_id = data.get('project_id')
    session_id = data.get("session_id") or session.get("session_id")

    if not session_id or session_id not in clients:
        return jsonify({'error': 'Invalid session'}), 400

    client_id = clients[session_id]["client"].client_id
    clients[session_id]["connection"].send(f"DELETE
projects#{client_id}.{project_id}")
    print(clients[session_id]["connection"].receive())

    # Update session
    session["projects"] = [p for p in session.get("projects", []) if
str(p['id']) != str(project_id)]

    return jsonify({'message': 'Project deleted'}), 200

@app.route("/project/<string:project_name>")
def project_dashboard(project_name):
    global sprints_data

    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

```

```

    if not session_id:
        return render_template("404.html", error="You made a mistake,
buddy"), 404
    project_name = project_name.strip() # Normalize input

    # Combine owned and shared projects
    all_projects = session.get("projects", []) +
session.get("shared_projects", [])
    print("All projects (normalized):", [p["name"].strip().lower()
for p in all_projects])

    # Find project (case-insensitive and trimmed match)
    project = next(
        (p for p in all_projects if p["name"].strip().lower() ==
project_name.lower()),
        None
    )

    if project:
        print("Project found:", project) # Debugging

        project_id = project["id"]
        session["project_id"] = project_id

        try:
            # Get all sprints for the project
            sprints_data = get_sprints(project_id)
            session["sprints"] = sprints_data

            # Ensure sprints is a list (Jinja requires an iterable)
            if isinstance(sprints_data, dict):
                sprints_data = [sprints_data]

            print(session["sprints"])
        except Exception as e:
            print(f"Error fetching tasks for project {project_id}:
{e}")

            return render_template('404.html', error="Error")

        return render_template(
            "HomeTasks_.html",
            project_name= project_name,
            project_id= project_id,
            sprints= sprints_data,
            projects=session["projects"],
            shared_projects=session["shared_projects"],
            session_id = session_id
        )

    else:
        print(f"Project '{project_name}' not found in projects
list.")
        return render_template("404.html",

```

```

project_name=project_name), 404

# region ----- SPRINTS -----

def get_sprints(project_id):
    clients[session["session_id"]]["connection"].send(f"GIVE
{session['client_id']}.{project_id}#sprints")
    response = clients[session["session_id"]]["connection"].receive()

    print(f"Raw task data received: {response}")
    # Clean up bytes-like prefix if present
    if response.startswith("b'") or response.startswith('b"'):
        response = response[2:-1] # Remove the b'...' or b"..."
wrapper

    # Replace single quotes with double quotes for valid JSON
    response = response.replace("'", '"')

    # Replace Python-style `None` with JSON-style `null`
    response = response.replace("None", "null")

    # Check for empty or invalid data
    if not response.strip():
        print("Failed to receive sprints.")
        return "Error loading tasks", 500
    try:
        # Attempt to parse the sanitized task data as JSON
        sprints = json.loads(response)
    except json.JSONDecodeError as e:
        print(f"JSON decode error: {e}")
        print(f"Sanitized task data: {response}")
        return "Error parsing tasks", 500
    # sprints need to be a dictionary inside a list -> [ ]
    return sprints

# Update Sprint Color Route
import copy

# TODO: Add the sockeio.emit for update sprint
@app.route('/update_sprint', methods=['POST'])
def update_sprint():
    global sprints_data
    try:
        data = request.get_json() # Use get_json for safety
        sprint_id = data.get('SprintID')

        if sprint_id is None:
            return jsonify({"success": False, "error": "SprintID is
required"}), 400

```

```

        sprint = next((t for t in sprints_data if
int(t.get("SprintID", -1)) == int(sprint_id)), None)
        if sprint is None:
            return jsonify({"success": False, "error": "Sprint not
found"}), 404

        # Use deep copy to isolate the task list
        tasks = copy.deepcopy(sprint["Tasks"])
        sprint["Tasks"] = None
        updated_fields = {}

        if 'SprintColor' in data:
            sprint["SprintColor"] = data["SprintColor"]
            updated_fields["SprintColor"] = data["SprintColor"]
            print(f"Updated Sprint {sprint_id} color to
{data['SprintColor']}")

        if 'SprintName' in data:
            if len(data["SprintName"]) > 2:
                sprint["SprintName"] = data["SprintName"]
                updated_fields["SprintName"] = data["SprintName"]
                print(f"Updated Sprint {sprint_id} name to
{data['SprintName']}")
            else:
                sprint["SprintName"] = "too short"

        if updated_fields:
            # Ensure connection object exists and send update
clients[session["session_id"]]["connection"].send(f"UPDATE_SPRINT#{sp
rint_id}${json.dumps(sprint)}")

        # Restore the deep copied tasks list
        sprint["Tasks"] = tasks
        session["tasks"] = tasks
        # TODO: Add the sockeio.emit for update sprint
        return jsonify([sprint]) # Return updated sprint in expected
format

    except Exception as e:
        print(f"Error: {e}")
        return jsonify({"success": False, "error": str(e)}), 500

# TODO: Add the sockeio.emit for adding sprint
@app.route('/add_sprint', methods=['POST'])
def add_sprint():
    try:
        data = request.get_json()
        existing_colors = {sprint["SprintColor"] for sprint in
session["sprints"]}

```

```

# Generate a unique random color
import random
def generate_unique_color():
    while True:
        color = "#{:06x}".format(random.randint(0, 0xFFFFFF))
        if color not in existing_colors:
            return color

sprint_color = generate_unique_color()

sprint_id = str(len(session["sprints"]) + 1) # Simple auto-
increment for demo
new_sprint = {
    "SprintID": sprint_id,
    "SprintName": data.get("SprintName", f"Sprint
{sprint_id}"),
    "SprintColor": sprint_color,
    "Tasks": []
}
projectId = data.get("ProjectID")
print(new_sprint)
#
connection.send(f"ADD_SPRINT#{session["project_id"]}.{task_id}${taskN
ame}")

clients[session["session_id"]]["connection"].send(f"ADD_SPRINT#{sessi
on['client_id']}.{projectId}${json.dumps(new_sprint)}")

    session["sprints"].append(new_sprint)
    return jsonify({"success": True, "sprint": new_sprint})
except Exception as e:
    return jsonify({"success": False, "error": str(e)})

# TODO: Add the sockeio.emit for deleting sprint
@app.route('/delete_sprint/<sprint_id>', methods=['DELETE'])
def delete_sprint(sprint_id):
    print(f"Received request to delete sprint {sprint_id}")
    try:
        original_sprints = session.get("sprints", [])
        print("Original sprints:", original_sprints)

        # Convert all SprintIDs to strings for consistent comparison
        new_sprints = [s for s in original_sprints if
str(s["SprintID"]) != str(sprint_id)]

        if len(original_sprints) == len(new_sprints):
            print("Sprint not found")
            return jsonify({"success": False, "message": "Sprint not
found."}), 404

        session["sprints"] = new_sprints

```

```

print("Sprint deleted")

project_id = session.get("project_id")
print(f"project_id: {project_id}")
session_id = session.get("session_id")
connection = clients[session_id]["connection"]
if session_id in clients:
    connection.send(f"DEL_SPRINT#{project_id}.{sprint_id}")

    return jsonify({"success": True, "deletedSprintId":
sprint_id}), 200
except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

# endregion

#region Share With Friends

@app.route('/share_project', methods=['POST'])
def share_project():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    # Parse the JSON request body.
    data = request.get_json()
    if not data:
        return jsonify(success=False, error="Invalid JSON data."),
400

    project_id = data.get('project_id')
    friend_id = data.get('friend_id')
    client = clients[session_id]["client"]
    # Validate required parameters.
    if not project_id or not friend_id:
        return jsonify(success=False, error="Missing project_id or
friend_id."), 400

    # Retrieve the current user's friends list from the session.
    friends = session.get('friends') or client.friends or []
    # Check if the friend_id exists in the session's friends list.
    friend_found = any(str(friend['id']) == str(friend_id) for friend
in friends)
    if not friend_found:
        return jsonify(success=False, error="Friend not found in your
session."), 404

    client_id = clients[session_id]["client"].client_id

clients[session_id]["connection"].send(f"SHARE_PROJECT#{client_id}.{f
riend_id}.{project_id}")

```

```

    print(f"Project {project_id} has been shared with friend
{friend_id}.")

    return jsonify(success=True), 200

#endregion

# region ----- HOME   TASKS -----

import json

@app.route("/delete_task", methods=["POST"])
def delete_task():
    global client_id
    data = request.get_json()
    task_id = data.get('task_id')
    project_id = data.get('project_id') # Get project_id from the
frontend
    session['project_id'] = project_id
    session['task_id'] = task_id
    username = session["username"]
    print(task_id)
    sprint_name = ""
    # Locate the task
    for sprint in session.get("sprints", []):
        for task in sprint.get("Tasks"):
            if task["TaskID"] == int(task_id):
                sprint_name = sprint["SprintName"]
                sprint["Tasks"].remove(task)
                break

    clients[session["session_id"]]["connection"].send(f"DELETE
tasks#{session['project_id']}.{session['task_id']}")
    response = clients[session["session_id"]]["connection"].receive()

    if "successfully" in response:
        # Emit deletion to all clients
        socketio.emit("delete_task", {
            "project_id": project_id,
            "task_id": task_id,
            "deleted_by": username,
            "sprint_name": sprint_name
        }, room=f"project_{project_id}")
        return jsonify({'status': 'success', 'message': 'Task deleted
successfully'}), 200
    else:
        return jsonify({'status': 'error', 'message': 'Task not
found'}), 404

# TODO: Make sure the sockeio.emit works for adding task
@app.route("/add_task", methods=["POST"])

```



```
def add_task():
    new_task = request.json
    sprint_id = new_task.get("SprintID")
    task_name = new_task.get("TaskName", "Untitled Task")
    username = session["username"]
    connection = clients[session["session_id"]]["connection"]

    sprints_data = session.get("sprints", [])
    if isinstance(sprints_data, str):
        try:
            sprints_data = json.loads(sprints_data)
        except json.JSONDecodeError:
            sprints_data = []
    # Find the sprint matching the SprintID
    matching_sprint = next(
        (s for s in session.get("sprints", []) if str(s["SprintID"])
    == str(sprint_id)),
        None
    )
    if matching_sprint:
        session["tasks"] = matching_sprint.get("Tasks", [])
    else:
        session["tasks"] = [] # fallback

    sprint_name = ""
    if session["tasks"]:
        for sprint in session.get("sprints", []):
            print(f"Checking Sprint: {sprint}")
            if sprint.get("SprintID") == sprint_id:
                sprint_name = sprint["SprintName"]
                session["tasks"] = sprint.get("Tasks", [])
                print("Sprint tasks found:", session["tasks"])
                break

    task_id = 1 if not session["tasks"] else max(task.get("TaskID",
0) for task in session["tasks"]) + 1
    new_task["TaskID"] = task_id

    # Ensure default values
    new_task["StatusColor"] =
STATUS_COLORS.get(new_task.get("Status", "In Progress"), "#d0ebff")
    new_task["PriorityColor"] =
PRIORITY_COLORS.get(new_task.get("Priority", "Medium"), "#9ca3af")
    new_task["Progress"] = 0 # Default progress

    # Append the new task to session["tasks"]
    session["tasks"].append(new_task)
    for sprint in session.get("sprints", []):
        print(f"Checking Sprint: {sprint}")
        if sprint.get("SprintID") == sprint_id:
            sprint["Tasks"] = session["tasks"]
            print("Sprint tasks found:", session["tasks"])
            break
```

```

try:
    client_id = session.get("client_id")
    project_id = session.get("project_id")

    if not client_id or not project_id:
        raise ValueError("Missing client_id or project_id in
session.")

connection.send(f"ADD_TASK#{client_id}.{project_id}.{sprint_id}.{task
_id}${task_name}")
    response = connection.receive()
    print(f"Received response: {response}")

except Exception as e:
    print(f"Connection error: {e}")
    return jsonify({"status": "error", "message": "Failed to add
task"}), 500

socketio.emit("task_added", {
    "project_id": project_id,
    "new_task": new_task,
    "added_by": username,
    "sprint_id": sprint_id,
    "sprint_name": sprint_name
}, room=f"project_{project_id}")

return jsonify({"status": "success", "newTask": new_task})

@app.route("/update_task", methods=["POST"])
def update_task():
    try:
        session_id = session["session_id"]
        connection = clients[session_id]["connection"]
        if not connection.connected:
            return render_template("404.html",
                                error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404

        update_data = request.json
        task_id = update_data.get("ID")
        sprint_id = update_data.get("SprintID")
        task_name = update_data.get("TaskName", "").strip()

        project_id = update_data.get("project_id") or
session.get("project_id")
        username = session.get("username")
        if not task_id or not sprint_id or not project_id:
            return jsonify({"status": "error", "message": "Task ID,
Sprint ID, and Project ID are required"}), 400

        print("Update data received:", update_data)
        print("Session sprints:", session.get("sprints", []))

```

```
sprint_name = ""
task = None
# region Locate the task
for sprint in session.get("sprints", []):
    if int(sprint.get("SprintID")) == int(sprint_id):
        sprint_name = sprint["SprintName"]
        for t in sprint.get("Tasks", []):
            if int(t.get("TaskID")) == int(task_id):
                task = t
                break

if not task:
    return jsonify({"status": "error", "message": "Task not found"}), 404
# endregion

# region Updating
# Track changes for logging
change_log = []

# Validate and update task name
if task_name:
    if task["TaskName"] != task_name:
        change_log.append(f"Task name changed from '{task['TaskName']}' to '{task_name}'")
        task["TaskName"] = task_name

new_due_date_str = update_data.get("DueDate")
print(type(new_due_date_str))
if new_due_date_str:
    task = updatePlus(task, update_data)
# endregion

socketio.emit("task_updated", {
    "project_id": project_id,
    "sprint_id": sprint_id,
    "updated_task": task,
    "changes": change_log,
    "updated_by": username,
    "sprint_name": sprint_name
}, room=f"project_{project_id}")

print(f"\n\nUpdated Task: {task}\n\n")

# Update the session with the modified task
for sprint in session.get("sprints", []):
    if int(sprint["SprintID"]) == int(sprint_id):
        for i, task_in enumerate(sprint["Tasks"]):
            if int(task_in["TaskID"]) == int(task["TaskID"]):
                sprint["Tasks"][i] = task # Directly update
task
```

```

        session.modified = True
        break

    print("Change log:", change_log)

    # Print updated sprint tasks
    for sprint in session.get("sprints", []):
        print(f"Tasks in Sprint {sprint['SprintID']}:
{sprint.get('Tasks', [])}")

    # TODO: if the date is closed to the DueDate, the system
would add a notification to the user about it.

    # Notify the client through WebSocket
    if session.get("session_id") in clients:
        try:

clients[session["session_id"]]["connection"].send(f"UPDATE_TASK${task
}")

            response =
clients[session["session_id"]]["connection"].receive() # No timeout
parameter

            print("WebSocket response:", response)
        except Exception as e:
            print(f"WebSocket error: {e}")

            return jsonify({"status": "success", "updated_task": task,
"changes": change_log})
        except Exception:
            return render_template("404.html", error="You made a
mistake, buddy"), 404

def updatePlus(task, update_data):
    today = datetime.today()
    new_due_date_str = update_data.get("DueDate")
    current_due_date_obj = parse_due_date(task.get("DueDate"))
    new_due_date_obj = parse_due_date(new_due_date_str) if
new_due_date_str else None

    # Only update if DueDate is different
    if new_due_date_obj and current_due_date_obj != new_due_date_obj:
        task["DueDate"] = new_due_date_obj.strftime("%d/%m/%Y") #
Store in same format

    due_date = parse_due_date(task.get("DueDate")) # re-parse just
to be safe
    days_left = (due_date - today).days if due_date else None
    print("Days left =", days_left)
    # --- Priority --- #
    if not update_data.get("Priority"):
        task["Priority"] = infer_priority(due_date, today)
        task["PriorityColor"] = PRIORITY_COLORS.get(task["Priority"],
"#9ca3af")

```

```

elif task["Priority"] != update_data["Priority"]:
    task["Priority"] = update_data["Priority"]
    if "PriorityColor" in update_data:
        task["PriorityColor"] = update_data["PriorityColor"]

# --- Status --- #
if not update_data.get("Status"):
    task["Status"] = infer_status(due_date, today)
    task["StatusColor"] = STATUS_COLORS.get(task["Status"],
"#d1d5db")
elif task["Status"] != update_data["Status"]:
    task["Status"] = update_data["Status"]
    if "StatusColor" in update_data:
        task["StatusColor"] = update_data["StatusColor"]

# --- Progress --- #
if "Progress" in update_data:
    try:
        task["Progress"] = int(update_data["Progress"])
    except (ValueError, TypeError):
        task["Progress"] = 0
else:
    task["Progress"] = infer_progress(task["Status"], due_date,
today)
align_priority_with_status(task, days_left)
return task

# region Extra functions
def parse_due_date(due_str: Optional[str]) -> Optional[datetime]:
    if not due_str:
        return None
    try:
        return datetime.strptime(due_str.strip(), "%d/%m/%Y")
    except (ValueError, TypeError):
        return None
def infer_priority(due_date: Optional[datetime], today: datetime) ->
str:
    if due_date is None:
        return "Missing"

    days_left = (due_date - today).days
    if days_left <= 2:
        return "Critical"
    elif days_left <= 5:
        return "High"
    elif days_left <= 10:
        return "Medium"
    else:
        return "Low"
def infer_status(due_date: Optional[datetime], today: datetime) ->
str:
    if due_date is None:
        return "Future Plan"

```

```

days_left = (due_date - today).days
if days_left < 0:
    return "Stuck"
elif days_left <= 2:
    return "Working On It"
elif days_left <= 5:
    return "In Progress"
elif days_left <= 10:
    return "Future Plan"
else:
    return "Ready to Start"
def align_priority_with_status(task: dict, days_left: Optional[int]):
    status = task.get("Status")
    priority = task.get("Priority")

    if status == "Future Plan" and priority in ["Critical", "High"]:
        task["Priority"] = "Low"
        task["PriorityColor"] = PRIORITY_COLORS["Low"]
    elif status == "Stuck" and priority in ["Low", "Best Effort"]:
        task["Priority"] = "High"
        task["PriorityColor"] = PRIORITY_COLORS["High"]
    elif status == "Ready to Start" and priority == "Critical":
        if days_left is None or days_left > 2:
            task["Priority"] = "High"
            task["PriorityColor"] = PRIORITY_COLORS["High"]
def infer_progress(status: str, due_date: Optional[datetime], today:
datetime) -> int:
    if status == "Done":
        return 100
    elif status in ["Pending Deploy", "Waiting for Review"]:
        return 90
    elif status in ["In Tests", "Updating"]:
        return 75
    elif status == "In Progress":
        if due_date:
            days_total = (due_date - today).days
            if days_total > 0:
                return max(25, min(75, 100 - int((days_total / 10) *
100)))
        return 50
    elif status == "Working On It":
        return 30
    elif status == "Ready to Start":
        return 0
    elif status == "Stuck":
        return 10
    else:
        return 0
# endregion

# Color mappings for status and priority

```

```

STATUS_COLORS = {
    "Ready to Start": "#d0ebff",
    "In Progress": "#facc15",
    "Waiting for Review": "#cfe2ff",
    "Updating": "#dbeafe",
    "Pending Deploy": "#e8e4d3",
    "Done": "#d1fae5",
    "Future Plan": "#baf5e0",
    "Working On It": "#e9d5ff",
    "Stuck": "#fca5a5",
    "In Tests": "#d9f99d",
    "": "#d1d5db",
}

PRIORITY_COLORS = {
    "Critical": "#f87171",
    "High": "#facc15",
    "Medium": "#bfdbfe",
    "Low": "#d1fae5",
    "Best Effort": "#e5e7eb",
    "Missing": "#9ca3af",
}

# endregion

import json

def get_notifications():
    print("Notification_1")
    if "email" not in session or not session["email"]:
        # Retrieve user email if not in session
        session_id = session["session_id"]

    clients[session_id]["connection"].send(f"GET_USER_EMAIL#{session.get('client_id')}")
    response = clients[session_id]["connection"].receive()
    if "ENDED" in response:
        return render_template("404.html",
                               error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
    if not response or "User not found" in response:
        print("User not found in database.")
        return [] # Return empty list if user is not found

    session["email"] = response.strip
    print(f"Email for client {session.get('client_id')}
retrieved: {response}")

    # Retrieve pending requests (friends + workers)

clients[session["session_id"]]["connection"].send(f"GET_PENDING_REQUE

```

```

STS#{session['email']})
    response =
clients[session["session_id"]]["connection"].receive().strip()
    print("Raw received data Notification_2:", repr(response)) #
Debugging output

    try:
        data = json.loads(response) # ✓ Correct JSON parsing
    except json.JSONDecodeError as e:
        print(f"JSON decoding error: {e}")
        return []

    notifications = []

    # New structure: list with a single dictionary containing
    "friends" and "workers"
    if isinstance(data, list) and len(data) > 0 and
isinstance(data[0], dict):
        data_obj = data[0]

        # Handle friend requests
        if "friends" in data_obj and isinstance(data_obj["friends"],
list):
            for friend_request in data_obj["friends"]:
                if isinstance(friend_request, dict):
                    sender = friend_request.get('sender_username',
'Unknown')
                    sender_id = friend_request.get('sender_id',
'Unknown')
                    notif_type = friend_request.get('type',
'unknown')

                    message = f"You have a new {notif_type} from
{sender}"
                    print("Formatted notification message:", message)

                    friend_request["message"] = message
                    notifications.append(friend_request)

        # Handle worker requests
        if "workers" in data_obj and isinstance(data_obj["workers"],
list):
            for worker_request in data_obj["workers"]:
                if isinstance(worker_request, dict):
                    sender = worker_request.get('sender_username',
'Unknown')
                    sender_id = worker_request.get('sender_id',
'Unknown')
                    notif_type = worker_request.get('type',
'unknown')

                    message = f"You have a new {notif_type} from
{sender}"

```



```

        print("Formatted notification message:", message)

        worker_request["message"] = message
        notifications.append(worker_request)

    else:
        print("Unexpected response format.")
        return []

    print(f"Total notifications: {notifications}")
    return notifications

#region FRIENDS
# Store active users (Online users: {client_id: socket_session_id})
active_users = {}

@app.route("/get_friends", methods=["GET"])
def get_friends():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
    session.get("session_id")

    if not session_id:
        return render_template("404.html",
                               error="Hold on, buddy. Something is
off...\nPlease wait for the updates"), 404
    if "client_id" not in session:
        return render_template("404.html",
                               error="Hold on, buddy. Something is
off...\nPlease wait for the updates"), 404
    try:
        client_id = clients[session_id]["client"].client_id
    except Exception:
        return render_template("404.html",
                               error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
    connection = clients.get(session_id)["connection"]
    if not connection:
        return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404

    payload = json.dumps({"client_id": client_id, "active_users":
list(connected_users.keys())})
    if len(list(connected_users.keys())) > 1:
        connection.send(f"GET_FRIENDS#{payload}")
        response = connection.receive()
        if "ENDED" in response:
            return render_template("404.html",
                                   error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
        else:

```

```

        return jsonify({"friends": []})
# try:
#     if not session["friends"]:
#         if not response:
#             if not response:
# except

try:
    friends_data = json.loads(response)

    if not isinstance(friends_data, dict) or "friends" not in
friends_data:
        return jsonify({"error": "Invalid response format"}), 500

    friends_list = friends_data["friends"]
    if not isinstance(friends_list, list):
        return jsonify({"error": "Invalid response format"}), 500

    for friend in friends_list:
        friend["online"] = str(friend["id"]) in connected_users

    session["friends"] = friends_list
    clients[session_id]["client"].friends = friends_list
    return jsonify({"friends": [friend for friend in friends_list
if friend["id"] != client_id]})

except json.JSONDecodeError:
    return jsonify({"error": "Failed to parse JSON response"}),
500
except Exception as e:
    print(f"Unexpected error: {e}")
    return jsonify({"error": str(e)}), 500

@app.route("/send_friend_request", methods=["POST"])
def send_friend_request():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    if "client_id" not in session or "username" not in session:
        return jsonify({"error": "User not logged in"}), 401

    try:
        data = request.get_json()
        receiver_email = data.get("receiver_email")
        sender_id = clients[session_id]["client"].client_id
        sender_username = session["username"]

        print(f"✉ Sending friend request from {sender_id}
({sender_username}) to {receiver_email}")

        connection = clients.get(session_id)["connection"]

```

```

if not connection:
    print("✗ Database connection missing!")
    return jsonify({"error": "Database connection not
found"}), 500

connection.send(f"GET_USER_BY_EMAIL#{receiver_email}")
response = connection.receive()
if "ENDED" in response:
    return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404
if not response or "User not found" in response:
    return jsonify({"error": "User not found"}), 404

try:
    receiver_id, receiver_username =
response.strip().split("#")
except ValueError:
    print(f"⚠ Unexpected response format: {response}")
    return jsonify({"error": "Invalid response from
database"}), 500

    print(f"✓ Found receiver: ID={receiver_id},
Username={receiver_username}")

    # Ensure sender_id is valid before proceeding
    if not sender_id or sender_id == "undefined":
        return jsonify({"error": "Invalid sender_id"}), 400

    # Send friend request command
connection.send(f"ADD_FRIEND_REQUEST#{sender_id}.{receiver_id}")

add_response = connection.receive()
if "ENDED" in response:
    return render_template("404.html", error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404

if "Success" in add_response.strip():
    # Emit a WebSocket notification
    socketio.emit(f"friend_request_{receiver_id}", {
        "message": f"You have a new friend request from
{sender_username}!",
        "sender_id": sender_id,
        "sender_username": sender_username
    }, room=receiver_id)
    # session["notifications"] =

    return jsonify({"success": "Friend request sent!"})
else:
    return jsonify({"error": "Failed to send friend
request"}), 500

```

```

except Exception as e:
    print(f"⚠ Error in send_friend_request: {str(e)}")
    return jsonify({"error": "Internal Server Error"}), 500

@app.route("/accept_friend_request", methods=["POST"])
def accept_friend_request():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    if "client_id" not in session:
        print("Unauthorized request - No session found")
        return jsonify({"error": "User not logged in"}), 401

    try:
        data = request.get_json()
        sender_id = data.get("sender_id")
        status = data.get("status")
        receiver_id = clients[session_id]["client"].client_id

        if not sender_id or not status:
            return jsonify({"error": "Invalid request"}), 400

        print(f"Processing request from {sender_id} by
{receiver_id}")

        connection = clients.get(session_id)
        if not connection:
            return jsonify({"error": "Database connection not
found"}), 500
        password = data.get("password")

        if status == "accepted":
            if password != session.get("password"): # Verify session
password
                return jsonify({"success": False, "error": "Incorrect
password"}), 403

        connection.send(f"CONFIRM_FRIENDSHIP#{sender_id}.{receiver_id}")
        confirm_response = connection.receive()
        if "ENDED" in confirm_response:
            return render_template("404.html",
                                error="Hold on, buddy.
Something is off...\nPlease 'wait' for the updates"), 404
            if confirm_response.strip() == "Friendship confirmed":
                return jsonify({"success": "Friend request
accepted!"})
            else:
                return jsonify({"error": "Friendship confirmation
failed"}), 500

        elif status == "denied":

```

```

        return jsonify({"success": "Friend request denied!"})

    return jsonify({"error": "Invalid status"}), 400

except Exception as e:
    print(f"Error in accept_friend_request: {str(e)}")
    return jsonify({"error": "Internal Server Error"}), 500

@socketio.on("new_friend_request")
def notify_receiver(data):
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    connection = clients.get(session_id)["connection"]
    if not connection:
        return jsonify({"error": "Database connection not found"}),
500

    try:
        receiver_email = data["receiver_email"]
        connection.send(f"GET_USER_BY_EMAIL#{receiver_email}")
        response = connection.receive()
        if "ENDED" in response:
            return render_template("404.html",
                                error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
            if "User not found" in response.strip:
                return

        recipient_id = response.split("#")[0]

        # Notify recipient
        socketio.emit(f"friend_request_{recipient_id}", {
            "message": "You have a new friend request!"
        }, room=recipient_id)

    except Exception as e:
        print(f"⚠ Error in notify_receiver: {str(e)}")

@app.route("/pending_friend_requests", methods=["GET"])
def pending_friend_requests():
    # Pull session_id from URL first
    session_id = request.args.get("session_id") or
session.get("session_id")

    receiver_email = session.get("email")

    connection = clients.get(session_id)["connection"]
    if not connection:
        return jsonify({"error": "Database connection not found"}),

```

500

```

if not receiver_email:
    return jsonify({"error": "User not logged in"}), 401

# Send request to Network.py with proper JSON formatting
import json
payload = json.dumps({"email": receiver_email})
connection.send(f"GET_PENDING_REQUESTS#{payload}")

response = connection.receive()
if "ENDED" in response:
    return render_template("404.html",
                           error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
if not response: # Handle empty response safely
    return jsonify({"pending_requests": []})
response = response.strip()
try:
    pending_requests = json.loads(response) # Expecting a JSON
response
    return jsonify({"pending_requests": pending_requests})
except json.JSONDecodeError:
    return jsonify({"error": "Invalid response format"}), 500

# region socketio
# Global dictionary to store connected users
connected_users = {}

# This will track which users are in which project rooms
user_rooms = {}

@app.route("/connected_users")
def get_connected_users():
    """Returns the list of currently connected users."""
    return jsonify({"connected_users": list(connected_users.keys())})

@socketio.on("connect")
def handle_connect():
    client_id = session.get("client_id")
    if client_id:
        connected_users[client_id] = {
            "client_id": client_id,
            "session_id": request.sid,
            "status": "online"
        }
    current_active_users = list(connected_users.keys())
    socketio.emit("update_online_status", {"active_users":
current_active_users}, to="all")

```

```

        join_room(f"user_{client_id}")
        socketio.emit("update_online_status", {"id": client_id,
"online": True}, to="all")

@socketio.on("disconnect")
def handle_disconnect():
    """Handles user disconnection and removes them from the connected
list."""
    client_id = None
    for user_id, user_data in connected_users.items():
        if user_data["session_id"] == request.sid:
            client_id = user_id
            break

    if client_id:
        del connected_users[client_id] # Remove the user from
connected users

        # Notify other users that this user is offline
        socketio.emit("update_online_status", {"id": client_id,
"online": False}, to=None)

        print(f"✗ User {client_id} disconnected.")

project_rooms = []
join_project = []
@socketio.on("join_project")
def on_join_project(data):
    try:
        global project_rooms
        project_id = data.get("project_id")
        room = f"project_{project_id}"

        join_room(room) # Joins the Socket.IO room
        print(f"Client {request.sid} joined {room}")
        if not project_rooms:
            project_rooms = [{"project_id": project_id, "users":
[f"{session['client_id']}"]}]]
        else:
            for project in project_rooms:
                if project_id in project["project_id"]:
                    if not session['client_id'] in project['users']:
                        project['users'].append(session['client_id'])
                        print(f"{session['username']} has joined the
room!")

                # for i in project["users"]:

            emit("joined_room", {"room": room}) # <-- emit ONLY to the
sender!
    except Exception:
        return render_template("404.html", error="You made a

```

```
mistake, buddy"), 404

@socketio.on("leave_project")
def on_leave_project(data):
    """Handles users leaving a project room."""
    client_id = session.get("client_id")
    project_id = data.get("project_id")

    if client_id and project_id:
        room = f"project_{project_id}"
        leave_room(room)

        # Remove the room from the user's list of rooms
        if client_id in user_rooms and room in user_rooms[client_id]:
            user_rooms[client_id].remove(room)

        print(f"Client {request.sid} ({client_id}) left {room}")
        emit("left_room", {"room": room}, room=room)

@socketio.on("team_update")
def handle_team_update(data):
    sender = session.get("username", "Unknown")
    project_id = data.get("project_id") # get it from the incoming
message

    if not project_id:
        print("! No project_id provided with message.")
        return

    message_data = {
        "sender": sender,
        "message": data.get("message", "")
    }

    socketio.emit("team_update", message_data,
room=f"project_{project_id}")
    print(f"✉ Message broadcast to project_{project_id} from
{sender}")

@socketio.on('chat_message')
def handle_chat_message(data):
    message = data['message']
    emit('chat_message', {'message': message}, broadcast=True)

# endregion

# endregion

# region --- WORKERS ---
```



```

@app.route('/team_dashboard')
@role_required('Manager', 'Worker')
def team_dashboard():
    return render_template('team_dashboard.html')

def fetch_workers_for_manager(manager_id):
    connection = clients.get(session["session_id"])["connection"]
    if not connection:
        return []

    connection.send(f"GET_WORKERS#{manager_id}")
    response = connection.receive()
    if "ENDED" in response:
        return render_template("404.html",
                                error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
    try:
        workers = json.loads(response)
        if isinstance(workers, dict):
            workers = [workers]
            session["workers"] = workers
            print("workers: ", workers)
        return workers
    except json.JSONDecodeError:
        return []

@app.route('/my_workers')
def my_workers():
    if "client_id" not in session:
        return redirect(url_for("login"))

    manager_id = session["client_id"]
    connection = clients.get(session["session_id"])["connection"]

    if not connection:
        return jsonify({"error": "No active connection."}), 500

    connection.send(f"GET_WORKERS#{manager_id}")
    response = connection.receive()
    if "ENDED" in response:
        return render_template("404.html",
                                error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
    try:
        workers = json.loads(response)
        return render_template('my_workers.html', workers=workers)
    except json.JSONDecodeError:
        return "Error loading workers", 500

@app.route('/register_worker', methods=['POST'])
def register_worker():

```

```

username = request.form['username']
email = request.form['email']
password = request.form['password']
manager_id = request.form['manager_id'] # Select a manager from
dropdown!

session_id = str(uuid.uuid4())
if session_id not in clients:
    clients[session_id] = ConnectionWithDatabase(session_id)

connection = clients[session_id]
if not connection.connected:
    connection.connect()
if not connection.connected:
    return render_template('404.html', error="Database connection
failed.")

connection.send(f"SIGNUP_WORKER
{username}#{email}#{password}#{manager_id}")
response = connection.receive()
if "ENDED" in response:
    return render_template("404.html",
                           error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
print("response=", response)

if response and "successfully" in response:
    session["username"] = username
    session["email"] = email
    session["session_id"] = session_id
    session["client_id"] = response.split("ID:")[1] if "ID:" in
response else None

    return redirect(url_for('login'))

    return render_template('404.html', error="Registration failed.
Please try again.")

@app.route('/assign_project_to_worker', methods=['POST'])
def assign_project_to_worker():
    if session.get('role') != 'Manager':
        return jsonify({"error": "Unauthorized access."}), 403

    data = request.get_json()
    worker_id = data.get('worker_id')
    project_id = data.get('project_id')

    if not worker_id or not project_id or worker_id ==
session['client_id'] :
        return jsonify({"error": "Missing worker or project ID."}),
400

    connection = clients[session["session_id"]]["connection"]

```

```

connection.send(f"ASSIGN_PROJECT#{session['client_id']}.{worker_id}.{
project_id}")
response = connection.receive()
if "ENDED" in response:
    return render_template("404.html",
                           error="Hold on, buddy. Something is
off...\nPlease 'wait' for the updates"), 404
if "assigned" in response.lower():
    return jsonify({"success": True})
else:
    return jsonify({"error": response}), 500

@app.route('/add_worker', methods=['POST'])
def add_worker():
    data = request.get_json()
    email = data.get('email')

    if not email:
        return jsonify({'error': 'Email is required'}), 400

    if email == session["email"]:
        return jsonify({'error': 'Others email is required'}), 400
    session_id = session["session_id"]

    clients[session_id]["connection"].send(f"SEND_REQUEST_WORKER#{session
['client_id']}.{email}")

    return jsonify({'message': 'Worker added successfully'}), 200

@app.route('/accept_work_request', methods=['POST'])
def accept_work_request():
    data = request.get_json()
    sender_id = data.get('sender_id')
    # TODO: Process accepting the work request
    return jsonify({"success": True})

@app.route('/decline_work_request', methods=['POST'])
def decline_work_request():
    data = request.get_json()
    sender_id = data.get('sender_id')
    # TODO: Process declining the work request
    return jsonify({"success": True})

# endregion

@app.route('/reset_timer', methods=['POST'])
def reset_timer():
    if "username" in session:
        session["time_since_login"] = time.time()
    return '', 204

```

```
@app.route('/register', methods=['POST'])
def register():
    try:
        username = request.form['username']
        email = request.form['email']
        password = request.form['password']

        session_id = str(uuid.uuid4())
        connection = ConnectionWithDatabase(session_id)

        if not connection.connected:
            connection.connect()
        if not connection.connected:
            return render_template('404.html', error="Database
connection failed.")

        connection.send(f"SIGNUP {username}#{email}#{password}")
        response = connection.receive()
        if "ENDED" in response:
            return render_template("404.html",
                                error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
            print("response=", response)

        if response and "successfully" in response:
            return render_template('login.html', message="You have
registered successfully!")

        return render_template('404.html', error="Registration
failed. Please try again.")

    except Exception as e:
        import traceback
        print("Exception in /register:", e)
        traceback.print_exc()
        return render_template('404.html', error="Internal server
error during registration.")

@app.route('/logout')
def logout():
    session_id = session.get("session_id")
    if session_id and session_id in clients:
        connection = clients[session_id].get("connection")
        if connection:
            try:
                connection.close()
            except Exception as e:
                print(f"Error closing connection: {e}")
        clients.pop(session_id, None) # Clean up the client entry
safely

    keys_to_clear = [
        'username', 'password', 'session_id', 'client_id',
```

```

        'notification', 'time_since_login', 'friends', 'tasks',
        'project_id', 'projects', 'shared_projects', 'role', 'pic',
        'sprints', 'email', 'workers', 'project_id', 'task_id'
    ]
    for key in keys_to_clear:
        session.pop(key, None)

    # Redirect back to login
    return redirect(url_for('login'))
@app.route('/profile')
def profile():
    try:
        session_id = session["session_id"]
        connection = clients[session_id]["connection"]
        if not connection.connected:
            return render_template("404.html",
                                   error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
            return render_template('profile.html')
        except Exception:
            return render_template("404.html", error="You made a
mistake, buddy"), 404
@app.route('/listToDo')
def listToDo():
    try:
        session_id = session["session_id"]
        connection = clients[session_id]["connection"]
        if not connection.connected:
            return render_template("404.html",
                                   error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404
            return render_template('listToDo.html')
        except Exception:
            return render_template("404.html", error="You made a
mistake, buddy"), 404

@app.route('/404')
def page_404():
    return render_template('404.html')

@app.route('/become_manager')
def become_manager():
    return render_template('404.html', Error="Error")

@app.route('/settings')
def settings():
    try:
        session_id = session["session_id"]
        connection = clients[session_id]["connection"]
        if not connection.connected:
            return render_template("404.html",
                                   error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404

```

```

        return render_template('settings.html')
    except Exception:
        return render_template("404.html", error="You made a
mistake, buddy"), 404
@app.route('/upload-avatar', methods=['POST'])
def upload_avatar():
    try:
        session_id = session["session_id"]
        connection = clients[session_id]["connection"]
        if not connection.connected:
            return render_template("404.html",
                                error="Hold on, buddy. Something
is off...\nPlease 'wait' for the updates"), 404

        data_url = request.form['croppedImage']
        if not data_url:
            return "No image data", 400

        # Extract base64 part from data URL
        match = re.search(r'^data:image/(png|jpeg);base64,(.*)$',
data_url)
        if not match:
            return "Invalid image format", 400

        image_data = base64.b64decode(match.group(2))
        user_dir = f"static/profile_pics/{session['username']}"
        # Create the directory if it doesn't exist
        os.makedirs(user_dir, exist_ok=True)
        with open(f"{user_dir}/avatar.png", 'wb') as f:
            f.write(image_data) # replace image_data with your
actual image bytes

        connection.send(f"UPDATE_PROFILE#{session['client_id']}.{user_dir}/av
atar.png")
        return redirect('/profile')
    except Exception:
        return render_template("404.html", error="You made a mistake,
buddy"), 404

def start():
    socketio_thread = threading.Thread(target=run_socketio)
    socketio_thread.daemon = True # Allow Flask to exit cleanly
    socketio_thread.start()

    app.run(debug=True, use_reloader=False, threaded=True,
host='0.0.0.0', port=5000)

```

ConnectionWithDatabase.py

```
import socket
from Security.Encryption import *

ADDR = ('127.0.0.1', 8085)

class ConnectionWithDatabase:
    def __init__(self, user_id):
        self.user_id = user_id
        self.socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        self.connected = False
        self.enc = Encryption()
        self.connect()

    # region ===== KEY EXCHANGE
    =====
    def key_exchange(self):
        binPubKey = self.socket.recv(1024)
        pubKeyObj = RSA.import_key(binPubKey)
        cipher_rsa = PKCS1_OAEP.new(pubKeyObj)
        enc_aes_key = cipher_rsa.encrypt(self.enc.key)
        # Send AES key encrypted by public key
        self.socket.send(enc_aes_key)
    #endregion

    def connect(self):
        if self.connected:
            print(f"User {self.user_id} is already connected.
Skipping reconnect.")
            return # Prevent reconnecting
        try:
            self.socket.connect(ADDR)
            self.connected = True

            # Receive the RSA public key from the server
            self.key_exchange()

            print(f"User {self.user_id} connected to the database.")
            self.send("HELLO_")
        except socket.error as e:
            print(f"Connection failed for {self.user_id}: {e}")

    def send(self, message):
```

```

try:
    encrypted_message = self.enc.encrypt_message(message)
    message_length = len(encrypted_message).to_bytes(4,
byteorder='big')
    self.socket.send(message_length + encrypted_message)
except Exception as e:
    print(f"Error sending message: {e}")

def receive(self):
    try:
        message = self.socket.recv(8192)
        if not message:
            print(f"User {self.user_id} disconnected.")
            self.close() # Ensure proper disconnection
            return "ENDED"
        return self.enc.decrypt_message(message[4:]) # Skip
length prefix
    except Exception as e:
        print(f"Receive error for {self.user_id}: {e}")
        self.close() # Close socket on error
        return "ENDED"

def close(self):
    """Close the connection properly."""
    if self.connected:
        print(f"Closing connection for {self.user_id}.")
        self.send(f"END")
        self.connected = False
        try:
            self.socket.shutdown(socket.SHUT_RDWR) # Properly
shutdown the socket
        except Exception as e:
            print(f"Error shutting down socket: {e}")
        self.socket.close()

```

Network.py

```

import socket
import select

from Security.Asymmetric import Asymmetric
from Security.Encryption import *

ADDR = ('0.0.0.0', 8085)

class Network:
    def __init__(self, dataBase):
        self.dataBase = dataBase

```



```

        self.enc = Encryption()
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        self.server_socket.bind(ADDR)
        self.server_socket.listen(5)
        # self.server_socket.setblocking(False)

        self.clients = {} # Dictionary to store active clients
        print(f"Server started on {ADDR}")
        self.run_server()

# region ===== KEY EXCHANGE
=====
def key_exchange(self, client_socket):
    asym = Asymmetric()
    # Send RSA public key to the client
    client_socket.send(asym.binPubKey)
    # Receive the encrypted AES key from the client
    enc_aes_key = client_socket.recv(1024)
    aes_key = asym.cipher_rsa.decrypt(enc_aes_key)
    print("AES KEY=", aes_key)

    # Create a new Encryption instance for this client
    enc = Encryption()
    enc.setKey(aes_key)
    self.enc.setKey(aes_key)
    return enc
# endregion
def run_server(self):
    """Main loop to handle multiple clients using `select`."""
    self.sockets_list = [self.server_socket]

    while True:
        read_sockets, _, exception_sockets =
select.select(self.sockets_list, [], self.sockets_list)

        for sock in read_sockets:
            if sock == self.server_socket:
                client_socket, client_address =
self.server_socket.accept()
                # KEY EXCHANGE
                enc = self.key_exchange(client_socket)
                client_socket.setblocking(False)

                self.sockets_list.append(client_socket)

                self.clients[client_socket] = {
                    "address": client_address,
                    "key": enc.key,
                    "enc": enc
                }

```

```

        print(f"New connection from {client_address}")
    else:
        try:
            message = self.receive(sock)
            if message is None:
                continue

            print(f"Received: {message}")
            if message.startswith("HELLO_"):
                user_id = message.split("_")[1]
                print(f"User {user_id} successfully
authenticated.")
            else:
                self.handle_message(sock, message)

        except Exception as e:
            print(f"Error receiving data: {e}")
            self.disconnect_client(sock,
self.sockets_list)

    def handle_message(self, client_socket, message):
        import json
        """Process client requests for multiple types of commands."""
        # try:
        # region Clients
        if message.startswith("SIGNIN"):
            username, password = message.split(' ', 1)[1].split('#')

            success, client_id, role = self.dataBase.login(username,
password)
            print(f"Login result: {success}, Client ID: {client_id}")
            # Debug

            if success:
                response = f"User exists. ID:{client_id};ROLE:{role}"
                self.send(client_socket, response)

                if client_socket in self.clients:
                    self.clients[client_socket].update({
                        "name": username,
                    })
            else:
                self.send(client_socket, "Username not found")

        elif message.startswith("SIGNUP"):
            try:
                items = message.split(' ')[1].split('#')
                username, email, password = items
            except ValueError:
                self.send(client_socket, "Invalid signup format.")
                return

```

```

        success =
self.dataBase.tables["clients"].addClient(username, password, email)
        if not success:
            self.send(client_socket, "Username or email already
exists.")
            return
        self.send(client_socket, f"User registered
successfully!")

# endregion

# region Friends
elif message.startswith("ADD_FRIEND_REQUEST"):
    items = message.split('#', 1)[1].split('.', 1)
    sender_id = int(items[0])
    receiver_id = int(items[1])
    print(f"Processing friend request from {sender_id} to
{receiver_id}")
    email_data =
self.dataBase.tables["clients"].getUserEmailById(receiver_id)
    success =
self.dataBase.tables["clients"].addFriendRequest(sender_id,
email_data)

    if success:
        self.send(client_socket, "Success")
    else:
        self.send(client_socket, "Failed to send friend
request")

    elif message.startswith("GET_USER_EMAIL"):
        receiver_id = message.split("#",1)[1].strip()
        email_data =
self.dataBase.tables["clients"].getUserEmailById(int(receiver_id))
        if email_data:
            self.send(client_socket, f"{email_data}") # Send
back user ID and username
        else:
            self.send(client_socket, "User not found")

    elif message.startswith("GET_USER_BY_EMAIL"):
        receiver_email = message.split("#", 1)[1].strip()
        print(f"Looking up user by email: {receiver_email}")

        # Query the database for the user's ID based on email
        user_data =
self.dataBase.tables["clients"].getUserByEmail(receiver_email)

        if user_data:
            user_id, username = user_data # Unpack user ID and
username
            self.send(client_socket, f"{user_id}#{username}") #
Send back user ID and username

```

```

        else:
            self.send(client_socket, "User not found")

    elif message.startswith("GET_PENDING_REQUESTS"):
        import json
        data = message.split("#", 1)[1].strip() # Extract
payload and remove extra spaces
        try:
            # Try parsing as JSON first
            payload = json.loads(data)
            receiver_email = payload.get("email") # Use .get()
to prevent KeyError
            print(f"receiver_email: {receiver_email}")
        except json.JSONDecodeError:
            # If it's not JSON, assume it's a direct email
            receiver_email = data if "@" in data else None
            if not receiver_email:
                self.send(client_socket, json.dumps({"error":
"Invalid request format"}))
                return
            # Fetch pending friend requests from the database
            pending_requests =
self.dataBase.tables["clients"].getPendingRequests(receiver_email)

            self.send(client_socket, json.dumps(pending_requests)) #
Send back as JSON

    elif message.startswith("CONFIRM_FRIENDSHIP"):
        sender_id, receiver_id = message.split("#")[1].split(".",
1)

        try:
            success =
self.dataBase.tables["clients"].addFriend(int(sender_id),
int(receiver_id))
            if success:
                print(f"Friendship added: {sender_id} ↔
{receiver_id}")
                self.send(client_socket, "Friendship confirmed")
            else:
                print(f"Failed to confirm friendship between
{sender_id} and {receiver_id}")
                self.send(client_socket, "Failed to confirm
friendship")
        except Exception:
            print("ERROR: sender_id is undefined!")
            self.send(client_socket, "Failed to confirm
friendship")

    elif message.startswith("GET_FRIENDS"):
        import json
        # try:
        data = message.split("#", 1)[1] if "#" in message else

```

```

"{}"
        payload = json.loads(data) # Avoids parsing errors on
empty input
        receiver_id = int(payload.get("client_id", 0))
        active_users = set(map(int, payload.get("active_users",
[[]])) # Ensure conversion to integers
        if receiver_id == 0:
            raise ValueError("Invalid client_id received")
        friends_data =
self.dataBase.tables["clients"].listFriends(receiver_id,
active_users)
        print(friends_data)
        self.send(client_socket, json.dumps(friends_data)) #
Send back as JSON
        # endregion

        # region Workers
        elif message.startswith("GET_WORKERS"):
            manager_id = int(message.split('#')[1])
            workers =
self.dataBase.tables["clients"].get_workers(manager_id)
            print(workers)
            self.send(client_socket, json.dumps(workers))

            elif message.startswith("SEND_REQUEST_WORKER"):
                import json
                data = message.split("#", 1)[1] if "#" in message else
"{}"
                manager_id, worker_email = data.split('.', 1)
self.dataBase.tables["clients"].addWorkerRequest(int(manager_id),
worker_email)
                elif message.startswith("ADD_WORKER"):
                    import json
                    data = message.split("#", 1)[1] if "#" in message else
"{}"
                    manager_id, worker_email = data.split('.', 1)
self.dataBase.tables["clients"].addWorkerRequest(int(manager_id),
worker_email)
                # endregion

                # region Projects
                elif message.startswith("GIVE_PROJECTS"):
                    import json
                    id = message.lower().split('#')[1]

                    projects, shared_projects =
self.dataBase.tables["projects"].allProjects(int(id))
                    print(f"projects#{projects}\n/{shared_projects}")

                    self.send(client_socket,
f"projects#{json.dumps(projects)}/{json.dumps(shared_projects)}")

```

```

elif message.startswith("ADD_ITEM"):
    data = message.lower().split('#', 1)
    id = int(data[1])
    print(id)
    projects =
self.dataBase.tables["projects"].addProject("New Project", "Describe
your project here.", id)
    self.send(client_socket, f"projects#{projects}")

elif message.startswith("DUPLICATE_ITEM"):
    # data = message.lower().split('#', 1)
    ids = message.split('#', 1)[1].split('.', 1)
    print(ids)
    for _ in ids:
        _ = int(_)

    projects =
self.dataBase.tables["projects"].duplicateProject(ids[1], ids[0])
    self.send(client_socket, f"projects#{projects}")

elif message.startswith("SHARE_PROJECT") or
message.startswith("ASSIGN_PROJECT"):
    ids = message.split('#', 1)[1].split('.', 2)
    print(ids)
    for _ in ids:
        _ = int(_)
    self.dataBase.tables["projects"].addSharedProject(ids[0],
ids[1], ids[2])
    if "ASSIGN_PROJECT" in message:
        self.send(client_socket,
                    f"assigned successfully")

# endregion

elif message.startswith("UPDATE "):
    import ast
    try:
        items = message.split(' ', 1)
        information = items[1].split("$", 1)
        subject = information[0].strip() # e.g., "projects"
        print("Subject:", subject)

        ids = information[1].split('#', 1)[0].split('.')
        client_id = int(ids[0])
        project_id = int(ids[1])

        changes =
ast.literal_eval(items[1].split('#')[1].strip()) # Correct parsing
        print("Changes: ---\n", changes)

        if subject == "projects":
            name = changes.get("name", "")
            desc = changes.get("description", "")

```

```

        print(
            f"Updating project with Name: {name},
Description: {desc}, Client ID: {client_id}, Project ID:
{project_id}")

        if subject not in self.dataBase.tables:
            print("Available tables:",
self.dataBase.tables.keys()) # Debugging step
            raise KeyError(subject)

self.dataBase.tables[subject].update_project(project_id, name, desc,
client_id)

        elif subject == "tasks":
            task_details = ast.literal_eval(changes) # Safer
dictionary parsing

            list_items = [
                int(task_details.get("TaskID", 0)),
                task_details.get("TaskName", ""),
                task_details.get("Priority", ""),
                task_details.get("Status", ""),
                task_details.get("StatusColor", ""),
                task_details.get("PriorityColor", ""),
                task_details.get("Owner(s)", ""),
                task_details.get("DueDate", ""),
                task_details.get("Tags", ""),
                int(task_details.get("Progress", 0)),
                project_id
            ]

self.dataBase.tables["tasks"].update_item(list_items)

            self.send(client_socket, "Item updated successfully")

        except (IndexError, ValueError, KeyError) as e:
            print(f"Error: {e}")
            self.send(client_socket, f"Failed to update item:
{e}")

        except Exception as e:
            print(f"Unexpected error during update: {e}")
            self.send(client_socket, "Failed to update item due
to an unexpected error.")

        elif message.startswith("DELETE"):
            data = message.lower().split('#', 1)
            ids = data[1].split('.')
            ids = [int(i) for i in ids]
            subject = data[0].split(' ')[1]

```

```

self.dataBase.delete(subject, ids)
self.send(client_socket, f"The {subject[: -1]} has deleted
successfully")

elif message.startswith("GIVE"):
    items = message.lower().split(' ', 1)[1].split('#')
    ids = items[0].split('.')
    c_id = int(ids[0])
    p_id = int(ids[1])
    name = items[1]
    list_ = self.dataBase.read_list_by_ID(c_id, p_id, name)
    self.send(client_socket, f"{list_}")

#region Sprints

elif message.startswith("UPDATE_SPRINT"):
    import json
    items = message.split('#', 1)
    print(items)

    data = items[1].split("$", 1)

    sprint_details = json.loads(data[1])
    print(sprint_details)
    ID = int(sprint_details.get("SprintID", 0)) # Convert to
integer

    SprintName = sprint_details.get("SprintName", "")
    SprintColor = sprint_details.get("SprintColor", "")

    list_items = [
        ID, SprintName, SprintColor
    ]

    self.dataBase.tables["tasks"].update_sprint(list_items)

elif message.startswith("ADD_SPRINT"):
    import json
    items = message.split('#', 1)
    print(items)

    data = items[1].split("$", 1)

    ids = data[0].split('.')
    print(ids)
    c_id = int(ids[0])
    p_id = int(ids[1])
    # item = data[1].strip("{}").strip()
    # sprint_id = int(ids[0])

    task_details = json.loads(data[1])
    # print(task_details)
    ID = int(task_details.get("SprintID", 0)) # Convert to
integer

```



```

        SprintName = task_details.get("SprintName", "")
        SprintColor = task_details.get("SprintColor", "")

        list_items = [
            ID, SprintName, SprintColor
        ]
        print(list_items)

self.dataBase.tables["tasks"].add_sprint_with_task(list_items, c_id,
p_id)

        elif message.startswith("DEL_SPRINT"):
            p_id, s_id = message.split('#', 1)[1].split('.')
            print("DELETE SPRINT")
            self.dataBase.tables["tasks"].delete_sprint(int(s_id),
int(p_id))

        #endregion

        # region Tasks
        elif message.startswith("UPDATE_TASK"):
            import json

            data = message.split("$", 1)

            # Fix JSON formatting issues
            item = data[1].strip() # Remove leading/trailing spaces
            item = item.replace("'", '"') # Convert single to double
quotes for JSON
            item = item.replace(": None", ": null") # Replace Python
None with JSON null

            print("Fixed JSON string:", item) # Debugging

            try:
                # Convert fixed string to dictionary
                task_details = json.loads(item)
            except json.JSONDecodeError as e:
                print(f"Error decoding JSON: {e}")
                task_details = {} # Fallback to an empty dictionary

            print("Parsed task_details:", task_details)

            # Extract values from task_details dictionary
            ID = int(task_details.get("TaskID", 0)) # Convert to
integer

            TaskName = task_details.get("TaskName", "")
            Status = task_details.get("Status", "")
            status_color = task_details.get("StatusColor", "")
            Priority = task_details.get("Priority", "")
            priority_color = task_details.get("PriorityColor", "")
            Owners = task_details.get("Owners", "") # Fixed: No need
for ('Owner(s)')

```

```

        DueDate = task_details.get("DueDate")
        Tags = task_details.get("Tags", "")
        Progress = int(task_details.get("Progress", 0)) #
Convert to integer

        if DueDate == "null": # Convert string null to Python
None
            DueDate = None

        # Create a list of items to update the database
        list_items = [
            ID, TaskName, Priority, Status, status_color,
            priority_color, Owners, DueDate, Tags, Progress
        ]
        # print("Final list_items:", list_items)

        # Update the database
        self.dataBase.tables["tasks"].update_item(list_items)
        print("Task updated successfully:", list_items)

        # Send confirmation back through the socket
        self.send(client_socket, f"Task no.{ID} has been
updated")

    elif message.startswith("DEL_TASK"):
        item = message.split(' ', 1)[1]
        print(item)
        self.dataBase.del_list_item(item)
        self.send(client_socket, f"{item} has deleted")

    elif message.startswith("ADD_TASK"):
        items = message.split('#', 1)[1].split('.')
        print(items)

        t_id, taskName = items[3].split('$', 1)
        c_id = int(items[0])
        p_id = int(items[1])
        s_id = int(items[2])
        t_id = int(t_id)
        self.dataBase.addTask(c_id, p_id, s_id, t_id, taskName)

        self.send(client_socket, "New empty task has created")

    # endregion

    elif message.startswith("END"):
        print("ENDDDDDE")
        self.disconnect_client(client_socket, self.sockets_list)
    else:
        self.send(client_socket, "Unknown command")

```

```

def send(self, client, message):
    try:
        if client:
            # Encrypt the message
            # self.enc.key = key
            enc = self.clients[client]["enc"]
            encrypted_message = enc.encrypt_message(message)
            total_length = len(encrypted_message)
            total_length_bytes = total_length.to_bytes(4,
byteorder='big')
            client.send(total_length_bytes+encrypted_message)
        else:
            print("Socket not connected. Attempting to
reconnect...")
    except Exception as e:
        print(f"Error sending message: {e}")

def receive(self, sock):
    try:
        raw_length = sock.recv(4) # Read first 4 bytes (message
length)
        if not raw_length:
            self.disconnect_client(sock, self.sockets_list)
            return None

        message_length = int.from_bytes(raw_length,
byteorder='big')
        buffer = sock.recv(message_length) # Read full message
        if not buffer:
            self.disconnect_client(sock, self.sockets_list)
            return None
        # Check if the message is encrypted (must be at least 16
bytes)
        enc = self.clients[sock]["enc"]
        if len(buffer) >= 16:
            try:
                message = enc.decrypt_message(buffer)
            except ValueError as e:
                print(f"Decryption error: {e}")
                return None # Ignore if decryption fails
        else:
            message = buffer.decode('utf-8') # Handle plaintext
messages

        return message
    except Exception as e:
        print(f"Receive error: {e}")
        self.disconnect_client(sock, self.sockets_list)
        return None

def disconnect_client(self, client_socket, sockets_list):
    """Disconnect a client."""

```

```
try:
    print(f"Client {self.clients[client_socket]['name']} or
client_socket} disconnected.")
    sockets_list.remove(client_socket)
    del self.clients[client_socket]
    client_socket.close()
except Exception as e:
    print(e)
```